La Torre de Morvath — Plan técnico de implantación de los pisos II a XIII y la cima

Documento de traspaso para el próximo Claude (o cualquier desarrollador) que continúe el proyecto. Redactado el 30 de septiembre de 2026 sobre el commit `d4564a0` de `main`. Acompaña a `ContextWindow/GUIA_PISOS.md`, que es la versión pedagógica (qué enseña cada piso). Este documento explica **cómo** construirlo sobre lo que ya existe.

Estado actualizado (30-09-2026, noche): pisos II a VIII jugables (II a VI subidos en ba920ec)

Lo hecho desde que se redactó este plan (léelo antes que el resto):

- **Modo equipo entre redes reales:** servidor de retransmisión TURN de Cloudflare (gratis, 1.000 GB/mes) a través del Worker `servidor-turn/` (`https://torre-morvath-turn.servidor-turn.workers.dev/credenciales`). La clave es un secreto del Worker (`wrangler secret put`); `js/config-red.js` solo tiene la URL. Donde este plan dice "sin TURN", ya no es así.
- **Fase 0, en parte.** Lo necesario para el Piso II sin romper el Piso I:
  - `js/nucleo/sala-torre.js`: `crearSalaDeTorre()` con origen desplazado y contrato de zona, y `cargarPiezasTorre()` con caché.
  - `js/mecanicas/logica.js`: lógica pura (regla 3-2-1, corrección por tipo de pregunta, serie de encargos).
  - `ui.quiz` admite los tipos `numero` y `orden`.
  - Carga diferida de pisos (`PISOS` y `asegurarPiso()` en `main.js`), con acciones `mec` que llegan antes de construir el piso guardadas en `pendientes`.
  - Acción genérica `mec` (validar/clave/aplicar del piso) y acción `subir`.
  - `LECCIONES` y `BANCO` reúnen todos los pisos; repaso espaciado en la varita.
  - `record(ok, criterio)` y `state.criterios`.
  - Progreso guardado versión 2, con migración.
  - Botón "Continuar en el Piso II" (también con `?docente=1`).
  - `herramientas/validar_contenido.mjs` y `herramientas/pruebas/*.test.mjs`.
- **Informe para el profesor** (`js/informe.js`, módulo puro con pruebas en `informe.test.mjs`): botón en la portada (si hay progreso guardado) y en la pantalla de fin. Pide el nombre y descarga una página HTML imprimible con los pisos superados (mejor precisión y tiempo) y los 13 criterios con aciertos, fallos, porcentaje y un nivel orientativo. Sale de `localStorage.torreMorvath`; no se envía a ningún sitio.
- **Entrar a mitad de partida** (o volver tras caerse), en los pisos II a VIII:
  - el anfitrión acepta el `hola` si `red.motivoNoTarde()` devuelve `null` (en la casa, el exterior, el Piso I o la pantalla de fin se sigue rechazando);
  - le manda un `snap` con el piso donde está, las acciones aplicadas en ese piso (`registro`, sin `subir` ni `fin`) y `state.hecho`;
  - el que llega construye el piso y repite esas acciones con `ui.silencio = true` (sin diálogos ni avisos, las preguntas se dan por respondidas y no cuentan en el informe); las acciones nuevas que llegan mientras tanto esperan en `colaTarde`.
  - Tras repetir las acciones se llama a `jugadorFuera` con cada `quien` que ya no está en la sala (si no, un orbe cogido por alguien que se fue se quedaría flotando).
  - Probado con 3 pestañas en el Piso III: la tercera entró con 2 de 3 puertas, la inscripción resuelta y una palanca subida, y abrió la tercera puerta para todos.
- **Mapa de la torre** (el Portal de los pisos, sección 8.8): un pedestal con una torre en miniatura en la

casa de Aldric (`casa.mostrarMapa`), visible si hay algún piso desbloqueado (`pisosDisponibles()`, la misma regla que «Continuar en el Piso N»; con `?docente=1`, todos).
    - Tras activar el cristal, **E** abre la lista y la acción `irPiso` lleva a todo el equipo (da la varita, salta el exterior y el Piso I).
    - En equipo solo elige el anfitrión, con su progreso; los demás reciben un aviso. `irPiso` no se repite al entrar a mitad de partida.
    - Probado solo: del mapa al Piso III.
- **Misiones claras (2026-10-03):** el panel de la misión muestra arriba **Ahora** (la primera sin hacer), el **Cómo** (la misma pista que H, `textoPista()`) y debajo la lista del piso. Una **guía en pantalla** (rombo ◆ con la distancia; flecha pegada al borde si queda fuera de la vista) apunta a `destinoActual()`: cada piso define `destino()` con el punto del sello que toca, o `'salida'` para la escalera. Casa, exterior y Piso I los calcula `main.js`.
- **Cinemáticas (`js/cinematica.js`, 2026-10-03):** mientras `cine.activa`, `tick()` solo llama a `cine.update()` (cámara por planos con `encuadre()`, actores con `Animador`, luz de relleno que sigue al encuadre). Bandas negras, subtítulos con nombre, rótulos; E/Espacio/clic pasan, **Esc salta** (fundidos instantáneos al saltar). Textos en `content.js` (`CINE_INICIO`, `CINE_TORRE`).
    - **Inicio** (sustituye a las páginas del prólogo): parque al atardecer en (800, 0, 0) con paletas claras → portal de runas bajo los cuatro amigos (Leo, Nayra, Dani y Aitor = encapuchado, pícara, caballero y bárbaro) → túnel violeta → salen por el portal de la casa → **Aldric de verdad** (aldric.glb sin holograma) furioso y luego curioso → montaje de los días siguientes → la noche del destello rojo → «La tercera mañana…». Corre dentro de `runFlow`, así que en equipo las acciones de otros esperan a que termine.
    - **Vista de la torre** al salir de la casa (relámpago y luz roja en la cima) y **rótulo de capítulo** («Piso II · La Bóveda…») en negro al entrar en cada piso (`irA(id, titulo)`).
    - Ojo al probar: en el panel del navegador en segundo plano no se dibujan fotogramas; usar `__torre.step(n)` y `__torre.cine`.
- **Base de datos:** propuesta en `ContextWindow/PLAN_SUPABASE.md` (no implementada).
- **Modo operador (`?operador=1`, 2026-10-05):** para revisar pisos sin pasárselos. **F2** (o el botón de la portada) abre un panel para saltar a cualquier zona con la varita y sin intros (`irOperador(id)`); **V** activa el **modo fantasma** (`ZONA_FANTASMA`: sin suelo, colisiones ni límites; Espacio sube, C baja, doble velocidad). Implica `?docente=1` (mapa con todos los pisos). Solo jugando solo; no guarda progreso.
- **Fluidez entre misiones (2026-10-03):**
    - al romper un sello, aviso grande «✦ Sello roto · n de 3» con **Siguiente: …** (`ui.anunciarSello`, desde `crearSellos` y el Piso I);
    - al abrirse la puerta, la cámara la enseña 2,8 s (`mostrarSalida()` → `cine.mostrar(desde, mirarA, s)`);
    - al llegar a un piso: rótulo de capítulo + plano desde lo alto de la sala (`presentarSala()`) antes de la intro;
    - en el Piso VIII, plano de la aparición de Morvath antes del duelo (`ctx.mostrar`).
    - Nada de esto se ejecuta con `ui.silencio` (al ponerse al día quien entra a mitad).
- **Gráficos (2026-10-03):** haces de luz suaves en las ventanas de las salas de torre (+~40 llamadas; Piso II: 235), viñeta CSS (`#vineta`), polvo al correr y al aterrizar, filtro de flashback en los diálogos «Recuerdo de la torre» (`body.recuerdo`) y etiquetas flotantes que se desvanecen si la cámara está a menos de ~2 m (`atenuarEtiquetas()`).
- **Pendiente de la Fase 0:**
    - Migrar el Piso I a `crearSalaDeTorre`; hoy sigue en `world.js` + `mazmorra.js`.
    - Extraer `nucleo/` de `main.js`.
- **Piso II · La Bóveda de la Memoria** (`js/pisos/piso2.js` + `js/contenido/piso2.js`), en el origen `(0, 0, −700)`:

    - Sello 1: altar de los soportes, con 9 pedestales y 9 encargos; hay que acertar 3 seguidos.
    - Sello 2: balanza de las unidades, con una pregunta de ordenar y 3 numéricas; cada jugador la hace en su pantalla y, si falla, sigue desde la prueba en la que se quedó.
    - Sello 3: cripta 3-2-1, con 3 orbes que se cogen y se dejan en 5 receptáculos (uno es la trampa "otra carpeta del mismo disco"), más una pregunta final sobre copias incrementales.
    - 15 preguntas de varita (criterio 1.1), música `boveda` y un recuerdo de Morvath: la biblioteca que ardió, que explica su motivo.
    - Probado solo (los 3 sellos, la salida y la pantalla de fin) y en equipo de 2: orbes cogidos a la vez, encargos y cripta sincronizados.
    - 285 *draw calls* (Piso I: 271).
- **Pendiente del Piso II:**
    - ~~Los opcionales~~ **Hecho** (2026-10-01): el **archivero** junto al armario del este (`archivero` en el contenido). Es individual y no hace falta para la puerta: lección `sistemasArchivos` (sin interrumpir a los compañeros), 4 casos (FAT32, exFAT, NTFS, ext4) con `serieDePreguntas`, lección `cifradoUnidad` (BitLocker, FileVault, LUKS, VeraCrypt) y una pregunta final; +20 de saber. Dos preguntas nuevas de varita (p2-16, p2-17). Los enunciados de las preguntas no admiten `**` (se verían los asteriscos).
    - ~~Una prueba con 3 jugadores y desconexión con un orbe en la mano~~ **Probado** (2026-10-01): al cerrar la pestaña de quien lo llevaba, el orbe vuelve al pedestal en ~8 s en las otras dos pantallas; al volver a entrar a mitad de partida ve los 3 orbes en el pedestal y puede coger uno.
- **Piso III · El Scriptorium Binario** (`js/pisos/piso3.js` + `js/contenido/piso3.js`), en el origen `(0, 0, −1400)`:
    - Sello 1: ocho palancas (el 128 a la izquierda vista desde la sala) y tres puertas en serie: 77, 0x3C y –5 en complemento a 2. La lección de complemento a 2 salta antes de la puerta roja. El atril muestra en vivo el valor sin signo, con signo y en hexadecimal.
    - Sello 2: inscripción con 4 preguntas (nuevo tipo `texto`: «Hola», código de la «M», UTF-8 de la ñ y el emoji) y la tabla ASCII en la pared.
    - Sello 3: relicario con 2 pesos (imagen de 6.220.800 B y audio de 31,75 MB) y luego 6 tomos que se llevan a las estanterías «con pérdida» o «sin pérdida».
    - Guiño: el espejo de las mallas pone al personaje en modo alambre y cuenta sus vértices y caras.
    - 14 preguntas de varita, música `scriptorium` y un recuerdo: Aldric y Morvath fueron amigos.
    - Unas 270 *draw calls*: las estanterías están fusionadas y los libros son una malla instanciada.
- **Motor común nuevo:**
    - `js/mecanicas/clasificar.js`: la mecánica genérica de coger y llevar objetos, con validación al soltar. Se reutilizará en los pisos VII y XIII.
    - `js/pisos/comun.js`: `crearSellos` (sellos, recuerdo y puerta), `serieDePreguntas` (progreso individual) y `cartelVivo`.
    - En `main.js`: `ORDEN_PISOS`, `siguientePiso` y `CONTENIDO_PISOS`. La salida de cada piso hace `subir` al siguiente y el último hace `fin`. La pantalla de fin toma el nombre del piso. El botón «Continuar en el Piso N» sale para el primer piso no superado cuyo anterior sí lo está.
- **Piso IV · Los Puentes Flotantes** (`js/pisos/piso4.js` + `js/contenido/piso4.js`), en el origen `(0, 0, −2100)`:
    - Es un **exterior**, sin `crearSalaDeTorre`: 7 islas hexagonales (Medieval Hexagon, en `assets/modelos/islas/`) más el muelle y el islote de la salida, unidas por pasarelas de piedra.
    - `suelo()` devuelve –200 fuera de islas y pasarelas: el jugador cae y reaparece en la última isla pisada. El arco de la salida (`arch_gate`) lleva las 3 runas.
    - Sello 1 (`conectar`): los enlaces de luz se tienden con E en un poste y E en otro; repetirlo quita el enlace. Tres fases:

    - Estrella: hojas biblioteca, taller y punto de acceso unidas solo al switch, y el switch al router.
    - Fibra: todo conexo y Tenerife–Gran Canaria unidas por fibra (primero una pregunta sobre el medio; con otra respuesta, el cable no se tiende).
    - Redundancia: `sinPuntoUnico` en el núcleo (router, switch, Tenerife y Gran Canaria).
    - Sello 2: `ui.formulario()` para 3 equipos con `validarEquipo()` y errores pedagógicos, y luego la explicación de DHCP.
    - Sello 3: `ui.terminal()` con el intérprete de `js/mecanicas/terminal.js` (ayuda, ipconfig, ping, tracert, reparar y cls). Al abrirla por primera vez, Morvath corta todos los enlaces de Gran Canaria; el anfitrión fija la lista en `validar`. Al reparar uno de ellos, la isla vuelve a responder, se rompe el sello y se restauran los demás.
    - `tracert` sin ruta llega hasta el router vecino que aún responde, porque los switches no aparecen en `tracert`.
    - 14 preguntas de varita (criterio 1.2), música `archipielago` y un recuerdo: la discusión «estrella con él en el centro» frente a «malla».
    - Unas 170 *draw calls*.
    - Probado solo de principio a fin, incluidos el cable equivocado, el formulario con errores, la terminal y la caída al vacío. **Sin probar en equipo.**
- **Módulos puros nuevos**, con pruebas en `herramientas/pruebas/redes.test.mjs`:
    - `js/mecanicas/grafo.js`: `conexo`, `ruta`, `esEstrella`, `puentes` (Tarjan), `sinPuntoUnico` y `subgrafo`.
    - `js/mecanicas/redes.js`: IPv4, máscaras, privadas, subred y `validarEquipo`.
    - `js/mecanicas/terminal.js`.
- **Opcionales del Piso IV** (hechos el 2026-10-01; individuales, no hacen falta para la salida, +20 de saber cada uno):
    - **Faro Wi-Fi** (la torre de Tenerife): lección `wifiCasa` y `ui.formulario` con SSID, seguridad y contraseña, validado por `validarWifi()` en `redes.js` (pruebas en `wifi.test.mjs`): rechaza abierta, WEP y WPA2, contraseñas de menos de 12 caracteres, con palabras típicas o con el SSID, y SSID con datos personales.
    - **Pozo de la chatarra** (el pozo del muelle): lección `raee` (RAEE, reutilizar/reparar/reciclar, obsolescencia programada y percibida, derecho a reparar) y 5 casos con `serieDePreguntas`. Se hizo como preguntas en vez de con `clasificar` para que sea individual.
    - Dos preguntas nuevas de varita (p4-15, p4-16). Ojo: en el Piso IV el `grupo` ya está en el origen del piso; lo que se añade al grupo va en coordenadas locales (`aMundo` es para el mundo: partículas y colisiones).
- **Aviso para scripts:** no escribáis código JS con `\n`, `\b` o `C:\` desde un *heredoc* de Bash con Python: los escapes llegan convertidos en caracteres de control. Escribid el script a un archivo.
- **Piso V · El Taller de los Heraldos** (`js/pisos/piso5.js` + `js/contenido/piso5.js`), en el origen `(0, 0, -2800)`:
    - `ui.editorWeb()`: dos `textarea` (HTML y CSS) y una vista previa en un `iframe sandbox=""` con `srcdoc`, más botones de 375, 768 y 1280 px, la lista de requisitos en vivo y el botón «Entregar».
    - Las imágenes `assets/web/{aldric,torre,boveda}.jpg` son capturas del juego y se inyectan como `data:` URL.
    - `js/mecanicas/web.js` es un analizador propio de CSS (reglas y `@media`), junto con colores, contraste WCAG y las comprobaciones de los tres sellos. El HTML se analiza con `DOMParser`, que no ejecuta nada.
    - Sello 1: cartel con h1, p, img con alt, lista de 3 o más elementos, enlace y 3 o más propiedades de CSS.
    - Sello 2: `@media (max-width)` e `img { max-width: 100% }`.
    - Sello 3: edicto de Morvath con 5 fallos: `lang`, `alt`, contraste 1,7:1, h1 → h4 y «haz clic aquí».

    - El primero que entrega rompe el sello; su título, su imagen y su primer párrafo se dibujan en un lienzo colgado en la pared norte para todos. Nunca se envía su HTML.
    - Lección opcional `ple` en el tablón con el `git log` real del juego.
    - 14 preguntas de varita (2.2 y 2.3, una de tipo `orden`), música `taller` y un recuerdo: los carteles de Morvath que nadie podía leer.
    - Unas 190 *draw calls*.
- **Error arreglado:** `luego: 'cartel'` en una lección de otro piso abría la pregunta del cartel del Piso I. Ahora solo se abre si la lección no lleva `piso`. **Usad nombres de `luego` únicos.**
- **Dependencias: plan para no necesitar ninguna** (el usuario tendría que aprobar cada una):
    - Piso VI: la llave se evalúa con muestreo analítico de las primitivas (caja, cilindro, esfera con unión o resta), sin `three-bvh-csg`. Las restas se ven como volúmenes rojos translúcidos.
    - Piso VIII: los *malware* son los personajes de Adventurers con paleta y tinte propios, sin descargar KayKit Skeletons.
    - Piso XI: los bloques se ejecutan paso a paso recorriendo su propio árbol JSON, sin `js-interpreter`.
    - Piso XIII: un mini-SQL propio (SELECT, WHERE, JOIN y UPDATE sobre tablas pequeñas) en vez de `sql.js`, o pedir aprobación.
- **Piso VI · La Forja de las Formas** (`js/pisos/piso6.js` + `js/contenido/piso6.js`), en el origen `(0, 0, −3500)`, **sin `three-bvh-csg`**:
    - `js/mecanicas/forja.js` (puro): primitivas caja o cilindro (eje x, y o z) que suman o restan, con `dentroModelo` analítico, `parecido` (IoU por rejilla de 0,1) y `estadisticas` (V, A, C).
    - `js/mecanicas/forjaVista.js`: una ventana con su propio `WebGLRenderer`. Incluye el plano de la llave, piezas con campos numéricos (posición y medidas), el molde translúcido, ejes X, Y y Z, el parecido en vivo, puntos de control (`CONTROLES`: el ojo hueco, los dientes macizos…) y la descarga STL con `STLExporter` de three/addons (solo las piezas que suman).
    - Sello 2: banco de medidas con una serie de preguntas (aristas del cubo, Euler, STL y laminado).
    - Sello 3: `js/mecanicas/elegir.js`, la **mecánica genérica «elegir con encargos»** (atril, pedestales en arco, racha de N aciertos), con programas de 3D.
    - Opcional: visor de realidades (lección RV, RA y RM). Horno con luz que late (8 luces en total).
    - Música `forja`, 12 preguntas (2.1) y un recuerdo: las gafas de Morvath que tapan lo que no le gusta.
    - Unas 220 *draw calls*.
- **Piso VII · La Gran Biblioteca** (`js/pisos/piso7.js` + `js/contenido/piso7.js`), en el origen `(0, 0, −4200)`:
    - Sello 1: `clasificar` con 10 obras y 8 estanterías: ©, dominio público/CC0, BY, BY-SA, BY-NC, BY-ND, software libre y propietario. Nuevas opciones de `clasificar`: `alCoger` (un aviso con el caso de la obra) y `escalonar` (etiquetas a dos alturas).
    - `crearEstanteria()` se ha movido a `js/pisos/comun.js`.
    - Sello 2: `abrirTablon()` (`js/mecanicas/bibliotecaVista.js`) con 5 pregones. Herramientas: lupa, fecha, autoría y verificador (hay que usar al menos una). Etiquetas: verdadera, bulo, sátira, clickbait y deepfake. Acción: compartir, no difundir, reportar o contrastar.
    - Sello 3: `abrirBuscador()` sobre un índice local de 20 documentos, con el analizador **puro** `js/mecanicas/buscador.js` («comillas», -palabra, site: y filetype:). Hay 3 retos en los que el libro tiene que salir el primero; las pruebas verifican que una búsqueda ingenua no basta. Después, una pregunta `orden` sobre las fases de la curación.
    - 13 preguntas de varita (2.3 y 3.3, una de tipo `texto`) y un recuerdo: los pregones falsos firmados con nombres ajenos.

    - Unas 230 *draw calls*.
- **Piso VIII · Las Criptas del Contagio** (`js/pisos/piso8.js` + `js/contenido/piso8.js`), en el origen `(0, 0, -4900)`, **sin KayKit Skeletons**:
    - Las criaturas son el Bárbaro y el Caballero de Adventurers, teñidos, sin varita, sombrero, capa ni casco y sin sombras.
    - Comportamientos:
    - virus: sale del cofre cuando alguien lo abre;
    - gusano: una copia más cada 7 s, hasta 3;
    - troyano: cofre de oro «¡Regalo gratis!» que se revela al acercarse;
    - ransomware: cofre encadenado con «Paga 100 monedas»;
    - spyware: da vueltas mirando al jugador, con un bocadillo «copiando teclas»;
    - botnet: un nigromante con 2 zombis.
    - Los paseos se calculan con el tiempo local de cada jugador (son decorativos). Abrir, revelar y desterrar son `mec` con clave única, así que desterrar a la vez cuenta una sola vez.
    - **Varita genérica en `main.js`:** una zona puede definir `dianas()` (objetos con `group.position`, `alive` y `data`) y `alApuntar(diana)`. `ui.scrollInfo` acepta `data.titulo` y `data.accion`.
    - Desterrar son 2 preguntas generadas: el tipo (entre 6) y la contramedida (entre 4).
    - Sello 2: `abrirMuralla()` (`js/mecanicas/murallaVista.js`) con 5 reglas de tráfico que permitir o denegar y 6 prácticas; la comprueba `revisarMuralla()` (lógica pura).
    - Sello 3: una serie de 3 preguntas de la tríada (confidencialidad, integridad y disponibilidad).
    - Opcionales: el pozo de la Wi-Fi pública y (2026-10-01) el **taller del parche** en la pared oeste: lección `actualizaciones` (individual), una pregunta de ordenar (copia → sistema → aplicaciones → reiniciar) y 3 de opción (avisos falsos de «actualiza aquí», fin de soporte, el router); +20 de saber. Música `criptas` y 13 preguntas de varita (3.1).
    - *Draw calls*: unas 230 desde la entrada y **hasta 305** con todas las criaturas a la vista (bajan al desterrarlas).
- **30-09 (noche) · el usuario pide NO hacer pisos después del VIII.** En su lugar: armas, enemigos variados y optimización.
    - **Armas por personaje** (`js/armas.js`, puro, y `js/ataques.js`), con la tecla R y las armas que ya traían los modelos:
    - Caballero: espada y escudo, tajo de 120°.
    - Bárbaro: hacha a dos manos, giro de 360°.
    - Pícara: ballesta, virote de 22 m.
    - Encapuchado: dagas gemelas (`dagger.gltf`), puñalada doble rápida.
    - La varita solo aparece al lanzar hechizos (`Player.tieneVarita`). Los compañeros ven el arma o la varita según `v`. Mensaje de red `ataque`.
    - Las zonas pueden definir `alGolpear(diana)`: aturde 3 s.
    - **Enemigos** (`js/enemigos.js`): 9 tipos, todos de KayKit (CC0) y del mismo estilo que el juego.
    - KayKit Skeletons: esbirro, guerrero, pícaro e hechicero, con `paleta_esqueleto_oscura.png` y los ojos que brillan.
    - Los 5 aventureros «corruptos» (caballero, bárbaro, pícara, encapuchado y mago), teñidos y sin armas, sombrero ni capa.
    - **El usuario probó los monstruos de Quaternius, los rechazó («horribles») y se borraron:** no usar estilos que no sean KayKit.
    - Todavía no hay jefe final con modelo propio.
    - **Guardianes** (`js/contenido/guardianes.js` y `js/mecanicas/guardianes.js`): 2 por piso del II al VII.
    - Patrullan sin hacer daño. R los aturde y F los destierra con una pregunta de repaso

(`ctx.preguntaRepaso`), que da +15 de saber.
- ○ Acción compartida `guardian` con clave única.
- ○ Se enganchan al piso en `ponerGuardianes()` de `main.js`, que añade `dianas`, `alApuntar` y `alGolpear` a la zona.
- ○ **Piso VIII**: sus criaturas usan ahora estos modelos:
- ○ virus = mago corrupto
- ○ gusano = esqueleto esbirro
- ○ troyano = esqueleto guerrero
- ○ ransomware = bárbaro corrupto
- ○ spyware = pícara corrupta
- ○ botnet = esqueleto hechicero con 2 caballeros corruptos
- ○ **Optimización:** `fusionarEstaticos()` en `sala-torre.js` funde el suelo, los muros, las ventanas, los estandartes y las columnas por material. Resultado en *draw calls*: pisos II a VII entre 90 y 210, Piso VIII 88 (antes 305).
- ○ La fusión también se aplica al Piso I (`mazmorra.js`, sin la hoja de la puerta: 277 → 117) al exterior (`exterior.js`: 281 → 50) y a la casa (`casa.js`, sin la hoja de la puerta: 95 → 56). Barrido del 2026-10-01 (llamadas por fotograma, con sombras y posproceso): exterior 50, Piso I 136, II 191, III 158, IV 154, V 96, VI 115, VII 144, VIII 126; ningún error en consola al cargar las 10 zonas. `fusionarEstaticos` conserva solo los atributos position, normal y uv (algunas piezas traen tangentes y no se podían fundir).
- ○ **Muebles** (`js/pisos/muebles.js`, KayKit Furniture Bits con `paleta_muebles_oscura.png`, en `assets/modelos/muebles/`):
- ○ alfombras teñidas de granate;
- ○ estanterías con libros en el Scriptorium;
- ○ cuadros en el Taller;
- ○ rincón de lectura en la Biblioteca;
- ○ un armario en la Bóveda.
- ○ **Jefe: la proyección de Morvath** al final del Piso VIII. Es el holograma de Aldric en rojo, con el modelo del mago y 5 escudos que orbitan. `crearSellos` admite `{ antesDePuerta }`.
- ○ Duelo sin castigo: una pregunta de cada piso (`jefe.preguntas`, del II al VIII); cada acierto rompe un escudo y un fallo trae una burla.
- ○ Al vencerle se desvanece, da +50 de saber y se abre la puerta.
- ○ Es individual: en equipo, cada jugador tiene su duelo (probado con 2 jugadores). La acción `fin` se encola con `runFlow` para no cortar un duelo que siga abierto.
- ○ **Menú de opciones** (`js/opciones.js`, tecla O; se guarda en `localStorage.torreMorvathOpciones`):
- ○ volumen de la música y de los efectos (`Sonido.ajustarVolumen`);
- ○ tamaño del texto (100-150 %, con la variable CSS `--texto` y `zoom`);
- ○ sensibilidad del ratón e inversión del eje vertical;
- ○ reducir el movimiento (sin sacudidas);
- ○ **calidad baja** para ordenadores lentos: resolución 1:1, sombras de 1024 en vez de 2048 y sin resplandor (bloom). Si los primeros 8 s de juego van a menos de 28 fps, un aviso sugiere activarla.
- ○ La cámara gira también con J y L, así que se puede jugar solo con el teclado.
- ○ Los diálogos y avisos llevan `aria-live`, así que un lector de pantalla los lee.
- ○ **Controles táctiles** (`js/tactil.js`, solo si `matchMedia('(pointer: coarse)')`):
- ○ joystick a la izquierda (al fondo del todo, corre);
- ○ arrastrar en el resto de la pantalla gira la cámara (el lienzo sigue un solo dedo por `pointerId`);

    - botones a la derecha: usar (E), atacar (R), varita (F), saltar, grimorio (G), pista (H), chat (T, solo en equipo) y opciones (O). Envían la tecla equivalente, así que reutilizan toda la lógica del teclado;
    - se ocultan en la portada y mientras hay un diálogo, pregunta o ventana abierta.
    - Probado con la emulación táctil del navegador; falta probarlo en una tableta de verdad.
    - **Probado en equipo de 2** (anfitrión e invitado) en los pisos III (palancas), IV (enlaces), VI (altar), VII (coger obras) y VIII (abrir el cofre): el estado coincide en los dos.
- **06-10-2026 · el usuario pide terminar el resto de pisos y un aspecto para el jefe final** (bucle de automejora, decisiones delegadas).
- **Piso IX · El Tribunal de los Datos** (`js/pisos/piso9.js` + `js/contenido/piso9.js`), origen `(0, 0, −5600)`, criterios 3.2 y 3.3:
    - Sello 1 (estrado, norte): lección `rgpd` y 6 juicios con `serieDePreguntas` (supresión, oposición, portabilidad, menores de 14, datos de salud, AEPD).
    - Sello 2 (cofre, oeste): lección `identidad`; qué clave para cifrar y para firmar y un `orden` del trámite en sede electrónica. Al abrirlo, **firma y verifica de verdad con WebCrypto** (ECDSA P-256; si `crypto.subtle` no existe, lo simula y lo dice).
    - Sello 3 (espejo, este): lección `huella`; la «sombra» enumera rastros **reales y locales** de la partida (nombre, lema, personaje, hora de inicio, aciertos, pantalla) y hay que decir si son huella activa o pasiva; después, el panel de privacidad.
    - 12 preguntas de varita (3.2 y 3.3), guardianes `g9a`/`g9b`, música `boveda`, 163 *draw calls*, probado solo de principio a fin (sellos, puerta y pantalla de fin).
- **Piso X · El Laberinto de las Sombras** (`js/pisos/piso10.js` + `js/contenido/piso10.js`), origen `(0, 0, −6300)`, criterio 3.3. **Tema sensible: `revisadoDocente: false`** (aviso en modo docente/operador hasta que un docente u orientador lo revise).
    - Laberinto: 3 muros interiores (`MUROS`, tramos `wall` con colisiones) y niebla densa (`zona.niebla` 0,075). La guía apunta al **hueco** siguiente si el objetivo está en otra franja (`porElLaberinto`).
    - Sombras: los modelos de Adventurers con material negro translúcido y ojos rojos (`Animador` en Idle).
    - Sello 1 (voces, oeste): lecciones `fraudes` y `grooming` (esta individual, al llegar a esos chats) y 4 **chats simulados** (`js/mecanicas/chatVista.js`): smishing, vishing, grooming y sextorsión. Elegir mal enseña la consecuencia y deja volver a elegir.
    - Sello 2 (ecos, este): lección `ciberacoso` y el chat del grupo de clase (ser buen testigo).
    - Sello 3 (reloj, norte): lección `bienestar` y 4 preguntas (descanso, control en la pareja, identidad positiva, difusión de imágenes).
    - **Botón «¿Necesitas ayuda? 017»** siempre visible en el piso (017 INCIBE, 900 20 20 10 ANAR, 112).
    - 11 preguntas de varita; el validador comprueba que cada elección de chat tenga una respuesta buena y consecuencias. 171 *draw calls*. Probado solo de principio a fin.
- **Piso XI · El Taller de los Autómatas** (`js/pisos/piso11.js` + `js/contenido/piso11.js`), origen `(0, 0, −7000)`, criterios 4.1 y 4.2. **Sin `js-interpreter`:** intérprete propio de bloques en `js/mecanicas/automata.js` (avanzar, girar, repetir, mientras no llegue, si hay muro delante; detecta muros, trampas y bucles infinitos; traduce a JavaScript y pseudocódigo), con pruebas en `automata.test.mjs` (que también comprueban que los niveles tienen solución y que el programa del depurador falla).
    - Editor (`automataVista.js`): tablero 2D, paleta de bloques, programa anidado (× quita; pulsar un bloque contenedor añade dentro; «Pasar a si no»), límite de bloques por nivel y el código en JS y pseudocódigo en vivo. Devuelve el programa del jugador, y el **autómata de latón 3D** del tablero del suelo lo repite.
    - Sello 1 (atril, oeste): lección `pensamiento` y 4 preguntas (símbolos del diagrama de flujo,

`orden` de un algoritmo, descomposición).
    - ○ Sello 2 (tablero, centro): lección `programacion` y 3 tableros (secuencia; bucle con ≤5 bloques; mientras + si con ≤4 bloques).
    - ○ Sello 3 (mesa, este): lección `depuracion`, un error de sintaxis (pregunta) y un programa con error de lógica que hay que corregir en el editor.
    - ○ 10 preguntas de varita. Probado solo de principio a fin con clics reales en el editor.
- **Piso XII · La Fragua del Código** (`js/pisos/piso12.js` + `js/contenido/piso12.js`), origen `(0, 0, −7700)`, criterios 4.3 y 4.4. **Sin Worker ni ejecutar código del alumno:** «código con huecos» (un desplegable por hueco) comprobado por `js/mecanicas/fragua.js` (pruebas en `fragua.test.mjs`), con la explicación de cada hueco mal puesto.
    - ○ Sello 1 (molde, oeste): lección `poo` (compilado/interpretado, IDE, Git, clases, objetos, herencia) y la clase `Criatura` / `Esqueleto extends Criatura` con 5 huecos (class, extends, super, new, saludar). Al acertar aparecen en la sala 3 criaturas con su nombre (Huesitos, Chispa, Tibia).
    - ○ Sello 2 (placa, pared norte): lección `dispositivos` (eventos, LED como datos, apps nativas/web/híbridas). Rejilla de 5 × 5 LED que genera en vivo `const RUNA = ["00100", …]`, 2 huecos (`alPulsar`, `mostrar`) y «Pulsar el botón A». La placa 3D de la pared muestra lo dibujado.
    - ○ Sello 3 (libro, este): lección `datos` (CSV, `split`, comentarios). El CSV al lado del código con 4 huecos (`//`, `split`, `','`, `+`); al acertar se «escribe» resumen.txt y una pregunta numérica sobre él.
    - ○ 10 preguntas de varita, música `forja`. Probado solo de principio a fin.
- **Piso XIII · El Observatorio del Oráculo** (`js/pisos/piso13.js` + `js/contenido/piso13.js`), origen `(0, 0, −8400)`, criterio 4.3. **Sin `sql.js`:** mini-SQL propio (`js/mecanicas/minisql.js`: SELECT con columnas o *, JOIN … ON, WHERE con AND, UPDATE … SET … WHERE; errores explicados) y clasificador del centroide más cercano (`js/mecanicas/oraculo.js`); pruebas en `observatorio.test.mjs` (incluye que el oráculo sesgado falla, que con ejemplos variados acierta y que «más de lo mismo» no lo arregla).
    - ○ Sello 1 (orbe, centro): lección `ia`; gráfico 2D (tamaño × brillo de ojos) con la frontera de decisión coloreada. Empieza sesgado (2 de 8); hay que añadir ejemplos representativos de la reserva hasta 7 de 8.
    - ○ Sello 2 (consola, oeste): lección `basesDatos`; tablas celdas y prisioneros y 3 retos (SELECT *, JOIN + WHERE para encontrar a Aldric en la **celda 7, la cima**, UPDATE … WHERE id = 7). Un UPDATE sin WHERE abre todas las celdas, lo explica y las vuelve a cerrar.
    - ○ Sello 3 (mapa, este): lección `bigData` (5 V, alucinaciones, Reglamento Europeo de IA, y que el juego se hizo con ayuda de IA) y 5 preguntas.
    - ○ 10 preguntas de varita, cúpula de estrellas y orbe con anillos. Probado solo de principio a fin con consultas escritas a mano.
- **La Cima · El Trono de Morvath** (`js/pisos/cima.js` + `js/contenido/cima.js`), origen `(0, 0, −9100)`, criterio 4.5 y repaso de los cinco bloques. **El juego ya se puede terminar de principio a fin y cubre los 13 criterios** (223 preguntas, 52 lecciones).
    - ○ Azotea al aire libre (zona propia, sin `crearSalaDeTorre`): suelo circular de radio 15, almenas fundidas en una malla, luna y estrellas; `suelo()`/`limitar()` circulares. `sinHolograma: true` oculta el holograma de Aldric (aquí está el de verdad, en una jaula).
    - ○ **Aspecto del jefe:** Morvath es el modelo del mago (`aldric.glb`, el reflejo oscuro de su amigo) con una **paleta propia negra y carmesí** generada por `herramientas/paleta_morvath.py` (`assets/texturas/paleta_morvath.png`), 2,7 m de alto, corona luminosa, aura roja y partículas. Sin descargas.
    - ○ Sello de la cima (visor, oeste): lección `realidad` (RV, RA, RM, XR, marcador, anclas, CoSpaces/Unity/A-Frame) y 4 preguntas; al terminar aparecen tres runas de puntos débiles sobre Morvath.

    - **Duelo** (individual en equipo, como el del Piso VIII): 5 escudos, uno por bloque: binario (pregunta numérica), bulo, phishing, **arreglar el autómata de Morvath** (editor del Piso XI) e IA con sesgo. Sin castigo: si se falla, una burla y se repite.
    - Final: Morvath se arrodilla, la jaula se abre, Aldric habla y `fin` muestra «La torre es libre · Fin» con **créditos** (KayKit CC0, Three.js, PeerJS, tipografías, IA).
    - 112 *draw calls*. Probado solo de principio a fin.
- **Automejora (06-10-2026):**
    - Música nueva según la sección 13: `tribunal` (IX, fa menor solemne), `sombras` (X, bordón grave y latidos), `observatorio` (XIII, pads agudos) y `cima` (pulso de tambor y acordes épicos). Tras vencer a Morvath vuelve a sonar `parque`: el camino a casa.
    - **Probado en equipo de 2 en la Cima:** el visor (lo rompe el invitado) se sincroniza y, al vencer el invitado a Morvath, el final y la pantalla «La torre es libre» llegan a los dos.
    - Barrido de llamadas de dibujo: IX 151, X 159, XI 156, XII 150, XIII 133, Cima 112; ningún error al cargar.
- **La salida del último piso construido lleva al final:** si `subir` apunta a un piso que no existe en `PISOS`, `main.js` lo cambia por `fin`. El Piso VIII ahora sube al IX.
- **Para añadir un piso nuevo:** crear `js/contenido/pisoN.js` y `js/pisos/pisoN.js` (copiar el Piso VII como plantilla), añadirlo a `PISOS` y `CONTENIDO_PISOS` en `main.js`, a `validar_contenido.mjs` y a `contenido/guardianes.js`, y apuntar la salida del piso anterior a `['subir', { piso: 'pisoN' }]`.
- La pantalla de fin aparece al terminar el último piso construido. Cada piso lleva al siguiente por la escalera.

Índice

1. Contexto, reglas del usuario y cómo arrancar
2. Estado actual: arquitectura del juego
3. Referencia de los módulos existentes (API)
4. El multijugador actual
5. Recursos 3D, paletas y licencias
6. Cómo se ha probado hasta ahora (y trampas conocidas)
7. Deuda técnica que hay que pagar antes de crecer
8. Fase 0 — El motor de pisos (refactor previo obligatorio)
9. Catálogo de mecánicas reutilizables
10. Especificación técnica piso a piso
11. Nuevas dependencias propuestas (verificadas)
12. Multijugador en los pisos nuevos
13. Sonido de los pisos nuevos
14. Evaluación, informe por criterios y privacidad
15. Accesibilidad, opciones y controles táctiles
16. Rendimiento en ordenadores de instituto
17. Pruebas
18. Publicación (GitHub Pages) y caché
19. Hoja de ruta por fases
20. Riesgos y decisiones abiertas

Apéndices: A) Matriz saber → piso → mecánica → criterio · B) Esquemas de datos · C) Catálogo de piezas KayKit · D) Mensajes de red · E) Lista de comprobación por piso · F) Comandos útiles

---

1. Contexto, reglas del usuario y cómo arrancar

1.1 Qué es el proyecto

La Torre de Morvath es un juego educativo 3D para el navegador (Three.js, sin compilación) de la materia **Informática y Digitalización** en Canarias. Unos alumnos son teletransportados a un mundo *dark fantasy* por el mago Aldric; Morvath lo secuestra y los alumnos suben su torre. Cada piso enseña y evalúa una parte del currículo. El holograma de Aldric enseña; la varita hace preguntas de repaso antes de lanzar hechizos.

- Carpeta: `C:\Users\fatys\Desktop\DarkFantasy-Reworn`
- Repositorio: `https://github.com/hamzaleghouiti12-sketch/DarkFantasyProyect` (rama `main`)
- Currículo de referencia: `ContextWindow/Informatica-Digitalizacion-II_conceptos.pdf` (2.º Bachillerato, Decreto 78/2025, Anexo 2). Resumen por pisos en `ContextWindow/GUIA_PISOS.md`.

1.2 Reglas del usuario (Hamza) — respetarlas siempre

1. **Español** en todo: textos del juego, comentarios de código, commits y respuestas.
2. **No puede gastar dinero.** Nada de servidores de pago, servicios de pago ni APIs de pago. Solo opciones gratuitas (por eso el multijugador usa el *broker* público de PeerJS).
3. **Aprueba cada descarga por enlace.** Antes de descargar un pack o una librería nueva, enseñar el enlace, la licencia y el tamaño y esperar su visto bueno. Las librerías cargadas por CDN en tiempo de ejecución se le mencionan igualmente.
4. **Estilo KayKit oscurecido** (opción B elegida por él): modelos de Kay Lousberg (CC0) con la paleta repintada por `herramientas/oscurecer_paleta.py`. Mantener una sola familia de estilo.
5. **Audio sin derechos de autor:** todo el sonido se sintetiza con Web Audio (`js/audio.js`). No incrustar música con copyright.
6. **Subir a GitHub solo cuando lo pida** (suele decir "súbelo"). Commits en español con la línea `Co-Authored-By` que indique el sistema.
7. **No tocar ni subir archivos suyos:** `Nuevo Archivo PY.py` y los cambios en `pitchdeck.txt` son suyos; no incluirlos en los commits.
8. Explicar en lenguaje llano; no es programador profesional, pero entiende bien los conceptos.

1.3 Cómo arrancar el juego

Los módulos ES no funcionan con `file://`; hace falta un servidor local:

- **Desde Claude Code:** `preview_start` con el nombre `juego-torre` (definido en `C:\Users\fatys\.claude\launch.json`, puerto 8750, `python -m http.server` sobre la carpeta del proyecto).
- **El usuario:** doble clic en `Jugar.bat` (abre `http://localhost:8750`).
- Dependencias en tiempo de ejecución (CDN, requieren internet): Three.js 0.170.0 (importmap en `index.html`), PeerJS 1.5.5 (solo al entrar en modo equipo), Google Fonts (Cinzel, Alegreya Sans).
- Herramienta auxiliar: Python 3 con Pillow para regenerar paletas; Node está instalado y sirve para `node --check` y pruebas de módulos puros.

---

2. Estado actual: arquitectura del juego

2.1 Flujo de partida

Portada → **Jugar solo** o **Jugar en equipo** (sala con código, 2-5 jugadores, elección de personaje y etiqueta, chat) → prólogo (4 páginas) → **casa de Aldric** (cristal → holograma → decisión: portal = final triste y volver a decidir; puerta = rescatar, entrega la varita) → **exterior** (camino, cementerio, cripta, santuario, comentarios de Aldric) → arco-portal → **Piso I** (3 sellos: cartel 2FA, altar de contraseñas, pergaminos de phishing con la varita) → puerta norte y escalera → pantalla de fin con estadísticas.

2.2 El truco de las zonas

Todo vive en **una sola escena** de Three.js. Cada zona es un `THREE.Group` colocado lejos de las demás y el jugador se "teletransporta" entre zonas con un fundido a negro:

Zona	Origen (offset)	Módulo	Grupo
Casa de Aldric	`(-400, 0, 0)`	`js/casa.js` (`CASA_O`)	`casa.grupo`
Exterior	`(+400, 0, 0)`	`js/exterior.js` (`EXT_O`)	`exterior.grupo`
Piso I	`(0, 0, 0)`	`js/world.js` + `js/mazmorra.js`	`grupoTorre`

`aplicarZona(z)` en `main.js` hace visible solo el grupo de la zona activa (así las luces de otras zonas no cuentan en los shaders), cambia niebla, fondo, `camera.far`, intensidad de luna y cielo, distancia de cámara, música y el nombre de la zona en el HUD. `colocarEn(z)` pone al jugador en `z.entrada` (desplazado lateralmente según su color en equipo). `irA(id)` combina ambos con fundido.

2.3 La interfaz de zona (contrato)

Cada zona es un objeto con esta forma (ver `world.zona` al final de `world.js`, `crearCasa` y `crearExterior`):

```
{
  id: 'piso1', nombre: 'Piso I · …', grupo: THREE.Group,
  colliders: [{ x, z, r, tag? }],          // círculos en coordenadas de MUNDO
  musica: 'torre' | 'casa' | 'exterior',   // capa de js/audio.js
  pisada: 'piedra' | 'madera' | 'hierba',  // sonido de pasos
  camDist: 6.5, lejos?: 230,                // distancia de cámara y camera.far
  niebla: { color, densidad }, luz: { luna, cielo },
  entrada: { pos: Vector3, mirada: rad, yaw: rad },
  suelo(p) → altura,                        // escaleras, desniveles
  limitar(p, prevZ),                        // paredes, puertas
  limitarCamara(c, jugador),
  guiaHolograma(destino, jugador),          // dónde flota Aldric
  update?(dt, t, fx, jugador),              // animaciones propias
}
```

El jugador (`Player.update`) y el holograma (`Hologram.update`) solo hablan con la zona a través de este contrato. **Todo piso nuevo debe cumplirlo.**

2.4 El bucle

`frame()` (requestAnimationFrame) → `tick(dt, t)` → render con `EffectComposer` (RenderPass → UnrealBloomPass(0.75, 0.55, 0.85) → OutputPass). Tone mapping ACES, sombras PCFSoft con **una sola luz direccional** (la luna) que **sigue al jugador** (`moon.position = jugador + (-14, 24, 8)`), así hay sombras nítidas en cualquier zona con un `shadow.camera` de ±24 m.

`tick` hace, en orden: entrada → `player.update` → `aldric.update` → sonidos de pasos → luna → `zona.update` (solo la activa) → hechizos, partículas y compañeros → envío de posición en red (12 Hz) → lógica de la fase `play` (cooldown de la varita, objetivo de la varita, interactuable más cercano, disparadores automáticos, fin) → cámara.

2.5 Flujos (`runFlow`) y "ocupado"

Toda secuencia con diálogos o preguntas se encola con `runFlow(fn)`: una cadena de promesas que garantiza que solo se ejecuta **una** secuencia a la vez. `busy()` = hay una secuencia en marcha o un modal abierto; mientras está ocupado el jugador no se mueve ni puede interactuar. Los eventos que llegan por la red también se encolan con `runFlow`, así nunca se pisan dos diálogos.

2.6 Estado global (`state` en `main.js`)

`phase` (`title` · `sala` · `story` · `play` · `end`), `learned` (Set de lecciones), `seals` (del Piso I), `saber`/`aciertos`/`fallos`, `varita`, `casa{…}`, `comentados`, `hecho` (acciones compartidas ya aplicadas), `pedido` (acciones pedidas pendientes de respuesta), etc. **Está pensado para un solo piso**; ver Fase 0.

---

3. Referencia de los módulos existentes (API)

Líneas aproximadas en el commit de referencia (4.592 líneas de JS en total).

Módulo	Líneas	Responsabilidad
`main.js`	1071	Orquestación: render, zonas, estado, flujos, desafíos del Piso I, varita, acciones compartidas, red, sala, cámara, bucle, depuración
`world.js`	417	Piso I hecho por código (respaldo), su lógica visual y `world.zona`
`audio.js`	354	Música y efectos sintetizados
`content.js`	302	Todo el texto: prólogo, casa, exterior, Piso I, banco de preguntas
`textures.js`	300	Texturas por código (piedra, ladrillo, madera, runas…) y `makeLabel`
`exterior.js`	289	Zona exterior
`ui.js`	277	Diálogos, preguntas, grimorio, historia, HUD, avisos, fundido
`red.js`	273	Modo equipo (PeerJS)
`casa.js`	210	Zona casa
`player.js`	209	Protagonista
`mazmorra.js`	168	Viste el Piso I con KayKit Dungeon
`hologram.js`	160	Aldric (shader holograma con *skinning*)
`modelos.js`	135	Carga de glTF, paletas, `Animador`
`companeros.js`	107	Avatares de otros jugadores
`particles.js`	92	Partículas aditivas
`chat.js`	87	Chat de equipo y filtro
`spells.js`	75	Hechizos de la varita
`personajes.js`	66	Personajes elegibles (plantillas)

3.1 `ui.js` (clase `UI`)

- `dialogue(lineas, orador?) → Promise`: caja de diálogo con escritura letra a letra; **negrita**; E/Espacio/Enter/clic para avanzar.
- `quiz(pregunta, rotulo) → Promise<boolean>`: pregunta de opción múltiple (baraja las opciones, teclas 1-4, explicación, "Continuar"). Pregunta: `{ id, concepto, texto, opciones[], correcta, explicacion }`. Llama a `onRespuesta(ok)`.
- `story(paginas, elecciones) → Promise<idElegido>`: pantalla de historia a pantalla completa.
- `openGrimoire(entradas)`, `toast(texto, tipo, ms)`, `prompt(texto)`, `setObjectives([{text, done}])`, `setSaber(n)`, `setWand(texto, lista)`, `setZona(nombre)`, `scrollInfo(datos)`, `showEnd(estadisticas)`, `fundido(negro) → Promise`, `hideScreen/showScreen`.
- `handleKey(e)`: si hay un modal abierto, su manejador (`keyTarget`) se come la tecla.
- `rich(texto)`: convierte **...** a `<strong>` escapando HTML.

3.2 `modelos.js`

- `cargarModelo(url)` (GLTFLoader), `cargarPaleta(url)` (flipY false, sRGB).
- `cargarPiezas(carpeta, nombres, paleta, ext='glb')` → `{ nombre: THREE.Group }` con la paleta aplicada.
- `colocador(grupo, piezas) → poner(nombre, x, y, z, ry=0, escala=1, sombra=true)`: clona (comparte geometría y material) y coloca.
- `aplicarPaleta`, `ocultar(root, nombres)`, `ajustarAltura(root, metros)`.
- `Animador(root, clips)`: `bucle(nombre, velocidad)`, `unaVezSolo(nombre, velocidad)`, `update(dt)`; expone `nombreBase`, `velocidadBase`, `nombreUnaVez`, `contador` (para la red).
- **Ojo:** GLTFLoader elimina los puntos de los nombres de nodo (`handslot.r` → `handslotr`).

3.3 `player.js` (clase `Player`)

Movimiento relativo a la cámara (4,2 m/s, 7 corriendo, salto 5,8, gravedad 16, radio 0,35). `update(dt, input, camYaw, zona, t)`; `usarModelo(modelo, clips)`; `mostrarVarita(bool)`; `tenir(hex)`; `estadoRed()`; `castAnim()`, `celebrar()`, `golpe()`; `wandTip(v)`; `ponerEtiqueta(grupoSprites)`. Banderas de un fotograma: `saltoAhora`, `aterrizaje`.

3.4 `personajes.js`

`PERSONAJES` (Encapuchado, Pícara, Caballero, Bárbaro), `configurarVarita(gltf, paleta, materialPunta)`, `cargarPersonaje(id) → Promise<{modelo, clips}>` (plantilla cacheada: quita armas de `handslotr/handslotl`, aplica paleta, 1,8 m, añade varita con nodo `punta`), `instanciar(plantilla)` (SkeletonUtils.clone y materiales propios).

3.5 `hologram.js`, `spells.js`, `particles.js`

- `Hologram`: `ShaderMaterial` con *skinning* (fresnel, líneas de escaneo, parpadeo); `usarModelo(gltf)`, `celebrar()`, `gesto()`, `update(dt, t, jugador, camara, hablando, zona)`. Se atenúa si la cámara lo atraviesa.
- `SpellSystem`: `cast(hechizo, desde, fnObjetivo|null, destinoFijo, alImpactar)`, `fizzle(pos)`, una sola `PointLight` reutilizada (no añadir luces dinámicas: recompila shaders).
- `Particles(escena, max, tamaño)`: `emit(pos, {count, color, intensity, speed, up, life, gravity, drag, jitter, dir})`. Hay dos sistemas: `fx.small` (3000) y `fx.big` (1500).

3.6 `audio.js` (clase `Sonido`)

`iniciar()` (tras un gesto del usuario), `alternarSilencio()` (tecla M), `ambientar('casa'|'exterior'|'torre')` con fundido de 3 s entre capas, y efectos: `paso(superficie, correr)`, `salto`, `aterrizaje`, `lanzar(hex)`, `impacto`, `chisporroteo`, `acierto`, `fallo`, `sello`, `puerta`, `cristalRoto`, `teletransporte`, `holograma`, `clic`, `mensaje`. Primitivas: `tono`, `ruido`, `campana`. Reverberación por convolución con respuesta de impulso generada.

3.7 `textures.js`

Generadores de canvas (`stoneFloorCanvas`, `brickCanvas`, `woodCanvas`, `bannerCanvas`, `parchmentCanvas`, `signCanvas`, `runeGlyphCanvas`, `runeCircleCanvas`), `canvasTex`, `glowTexture`, `rng(semilla)` y `makeLabel(texto, opciones)` (sprite de texto; se usa para etiquetas y bocadillos).

3.8 `content.js`

`PROLOGO`, `CASA`, `EXTERIOR`, `PISO1` (`intro`, `lecciones{id:{titulo, resumen, paginas, despues?}}`, `cartel`, `altar[]`, `pergaminos[]`, `pistas{}`, `sellosRotos[]`) y `PREGUNTAS[]` (12 preguntas con `concepto` ∈ `2fa|contrasenas|phishing`). La varita solo pregunta sobre conceptos aprendidos (`state.learned`).

3.9 Depuración

`window.__torre` expone: `state, player, world, casa, exterior, cam, ui, keys, sonido, aldric, red, companeros, zona (getter)`, `irA(id)` (sin fundido), `step(n, dt)` (avanza fotogramas aunque la pestaña esté oculta) y `capture(w, h)` (dataURL JPEG del canvas a resolución fija).

---

4. El multijugador actual

4.1 Topología

PeerJS 1.5.5 (MIT) se importa dinámicamente desde jsDelivr al entrar en modo equipo. El **anfitrión** registra en el broker público gratuito `0.peerjs.com` el id `torre-morvath-v1-XXXX` (4 letras sin I ni O). Los invitados se conectan a él por WebRTC (DataChannel fiable). **Estrella:** los invitados solo hablan con el anfitrión; el anfitrión reenvía. Máximo 5 jugadores (`MAX_JUGADORES`). No hay servidor TURN: redes muy restrictivas pueden impedir la conexión.

4.2 Mensajes

t	Dirección	Contenido
`hola`	inv → anf	nombre, personaje, etiqueta
`bienvenida` / `rechazo`	anf → inv	id asignado / motivo (llena, empezada)
`sala`	anf → todos	lista de jugadores `{id, nombre, color, personaje, etiqueta}`
`perfil`	inv → anf	cambio de personaje o etiqueta en la sala
`empezar`	anf → todos	arranca la historia
`pos`	todos (12 Hz)	`x,y,z,f`, animación base y de una vez, varita, zona
`hechizo`	todos	hechizo, origen, índice de objetivo, destino (cosmético)
`acc`	inv → anf	petición de acción compartida `{tipo, datos}`
`ev`	anf → todos	acción validada `{tipo, datos, autor}`
`chat`	todos	texto (el anfitrión pone nombre y color)
`latido`	todos (1 Hz)	"sigo aquí"

4.3 Acciones compartidas (autoridad del anfitrión)

En `main.js`: `accion(tipo, datos)` → si soy invitado, envío `acc`; si soy anfitrión (o juego solo), `valido()` → `ev` a todos → `aplicarEvento(tipo, datos, soyAutor)`. `claveAccion()` genera una clave única por acción (`sello:cartel`, `pergamino:3`…) que se guarda en `state.hecho` para no aplicarla dos veces. Tipos actuales: `cristal`, `salir`, `torre`, `leccion`, `sello`, `altar`, `pergamino`, `comentario`, `fin`. Lo individual (preguntas de la varita, estadísticas, portal de volver a casa) nunca pasa por aquí.

4.4 Robustez

- **Latido** cada segundo desde un **Web Worker** (los temporizadores de pestañas ocultas se frenan); 10 s sin señal = desconectado. Motivo: PeerJS no emite `close` si el otro cierra el navegador de golpe (ICE se queda en `disconnected`).
- El anfitrión se **re-registra** en el broker si pierde el contacto (`peer.reconnect()`), para que la sala se siga encontrando.
- Si cae el anfitrión, los invitados siguen **en solitario** (`red.activa = false` → `accion` pasa a local).
- Sala: no se entra con la partida empezada; "Empezar" exige ≥ 2 jugadores.

---

5. Recursos 3D, paletas y licencias

5.1 Qué hay en `assets/`

- `modelos/`: `protagonista.glb` (Rogue_Hooded), `picaro.glb`, `caballero.glb`, `barbaro.glb` (~3,5 MB cada uno por sus 76 animaciones), `aldric.glb` (Mage), `varita.gltf` + `wand.bin` + `mage_texture.png`.
- `modelos/mazmorra/` (30 GLB de Dungeon Remastered), `modelos/casa/` (14 GLB de Dungeon Remastered), `modelos/exterior/` (Halloween Bits y Medieval Hexagon en `.gltf + .bin` con sus texturas).
- `texturas/`: paletas originales y `*_oscura.png` (pícaro, mago, caballero, bárbaro, mazmorra, halloween, medieval).
- `licencias/`: LICENSE de cada pack (todos CC0 de Kay Lousberg).
- `_descargas/` (**fuera de git** por `.gitignore`): los 4 packs completos (Adventurers, Dungeon Remastered con 203 piezas, Halloween Bits, Medieval Hexagon). Se pueden volver a bajar de `github.com/KayKit-Game-Assets`.
- Total en git: ~22 MB de modelos.

5.2 Paletas oscuras

Todos los modelos KayKit toman su color de una textura de 8×4 franjas degradadas. `herramientas/oscurecer_paleta.py` clasifica cada franja por su color medio y la repinta entera según un perfil (`personaje`, `escenario`, `exterior`). **Para un pack nuevo:** copiar su textura a `assets/texturas/paleta_X.png`, añadir una línea a `TRABAJOS`, ejecutar `python herramientas/oscurecer_paleta.py` y revisar el resultado visualmente (hacer una imagen comparativa antes y después). Si una franja queda mal, ajustar la clasificación, no los modelos.

5.3 Escalas

- Dungeon Remastered: cuadrícula de **4 m** (muros de 4×4×1, suelo de 4×4); la escala es 1.
- Halloween Bits: escala 1 (valla de 2,2 m).
- Medieval Hexagon: son miniaturas (la torre mide 2,2 m); se escalan de ×6 (casa) a ×18 (torre) y ×30 (montañas).
- Personajes: `ajustarAltura(modelo, 1.8)`.

---

6. Cómo se ha probado hasta ahora (y trampas conocidas)

1. **Sintaxis:** `node --check js/archivo.js` (Node detecta módulos ES).
2. **Navegador integrado de Claude Code** (`mcp__Claude_Browser__*`): se abre el juego con `preview_start` y se ejecutan guiones con `javascript_tool` que pulsan botones, simulan teclas (`dispatchEvent(new KeyboardEvent('keydown', {code:'KeyE', key:'e'}))`) y avanzan el juego con `__torre.step(n)`.
3. **Capturas:** `__torre.capture(1280, 720)` devuelve un JPEG; para guardarlo en disco se usó un pequeño servidor Python que acepta POST en `http://127.0.0.1:8751/` y escribe en `capturas/` (el guion estaba en la carpeta temporal; recrearlo si hace falta, ~20 líneas con `http.server` y cabeceras CORS).
4. **Multijugador:** varias pestañas del navegador integrado (una por jugador) contra la misma URL; se han probado 2, 3 y 5 jugadores, partida completa, desconexiones del invitado y del anfitrión.

Trampas conocidas:

- **Caché del navegador:** el servidor Python no invalida la caché de los módulos. Antes de probar cambios: `for (const f of [...]) await fetch('/js/'+f+'.js', {cache:'reload'})` y luego recargar con `location.href='/?v=N'`.
- **Pestañas ocultas:** con el panel del navegador oculto no hay `requestAnimationFrame`; usar `__torre.step()`. Tras muchos minutos ocultas, Chrome **congela** la pestaña (ni scripts ni Workers); para pruebas largas abrir pestañas nuevas.
- **Los diálogos bloquean:** si una prueba "no mueve" al jugador, casi siempre hay un diálogo abierto o encolado (p. ej. la lección de phishing salta al acercarse a los pergaminos). Vaciar con un bucle que pulse E mientras `ui.open.dialogue`.
- **Los interactuables se recalculan en `tick`:** tras cerrar un diálogo hay que dar al menos un `step()` antes de pulsar E.
- **Scripts de Bash largos:** los heredoc de más de ~150 líneas han fallado en el Bash de esta máquina; escribir el guion Python a un archivo y ejecutarlo.
- El clasificador de permisos del entorno a veces falla de forma transitoria; no insistir más de 2-3 veces seguidas.

---

7. Deuda técnica que hay que pagar antes de crecer

1. **`main.js` es un monolito (1.071 líneas)** con lógica específica del Piso I (cartel, altar, pergaminos, varita apuntando a pergaminos, fin). Añadir 12 pisos así lo haría inmantenible.
2. **`world.js` + `mazmorra.js` construyen un piso concreto** con posiciones fijas; `ROOM` es un objeto global mutable (`mazmorra.js` cambia `H`, `puerta`, `escalera`, `salida`).

3. **`state` asume un único piso** (`seals`, `phishingTriggered`, `fraudLeft` como variable suelta).
4. **El contenido está en un solo archivo** y las preguntas no llevan criterio de evaluación.
5. **La varita solo sabe apuntar a pergaminos** (`pickTarget` mira `world.scrolls`).
6. **Solo hay preguntas de opción múltiple.**
7. **No hay pruebas automáticas** ni validación del contenido.
8. **Todo se carga al inicio** (casa, exterior, Piso I). Con 13 pisos la carga inicial y la memoria se dispararían.
9. **El progreso guardado** solo contempla el Piso I (`localStorage.torreMorvath`).
10. **Caché:** no hay versión en las URL de los módulos (problema al publicar).

La **Fase 0** (sección 8) resuelve 1-9 sin cambiar lo que ve el jugador.

---

8. Fase 0 — El motor de pisos (refactor previo obligatorio)

Objetivo: que añadir un piso sea escribir un archivo de contenido y un archivo de construcción, reutilizando mecánicas genéricas. **Criterio de hecho:** el Piso I migrado al motor nuevo se juega exactamente igual (solo y en equipo) y pasa la misma batería de pruebas que hoy.

8.1 Estructura de carpetas propuesta

```
js/
  nucleo/            ← lo que hoy está mezclado en main.js
    motor.js         render, bucle, cámara, entrada
    estado.js        estado global + progreso por piso
    flujos.js        runFlow, busy, esperar
    acciones.js      accion / valido / aplicarEvento genéricos + registro de manejadores
    zonas.js         aplicarZona, colocarEn, irA, carga diferida de pisos
    varita.js        preguntas, lanzar, objetivos (genérico)
    sala.js          interfaz de la sala de equipo
  pisos/
    registro.js      lista de pisos, offsets, orden, dependencias
    base.js          crearSalaDeTorre(): suelo, muros, columnas, antorchas, puerta con 3 runas, esca
    piso1.js … piso13.js, cima.js
  mecanicas/
    elegir.js  clasificar.js  ordenar.js  palancas.js  conectar.js  terminal.js
    formulario.js  editorWeb.js  modelado.js  enemigos.js  casos.js  cofreCripto.js
    bloques.js  codigo.js  placa.js  sql.js  oraculo.js  realidad.js
  contenido/
    comun.js         prólogo, casa, exterior
    piso1.js … piso13.js, cima.js
    preguntas.js     reúne el banco de preguntas de todos los pisos
  (los módulos actuales: ui, audio, player, hologram, spells, particles, textures,
   modelos, personajes, companeros, red, chat, casa, exterior se mantienen)
```

Hacerlo **por pasos pequeños**, con el juego funcionando tras cada commit: primero extraer `nucleo/` sin cambiar comportamiento; después `pisos/base.js` reproduciendo la sala del Piso I; después las mecánicas `elegir` (altar), `quiz` (cartel) y `dianas` (pergaminos); por último borrar `world.js` y `mazmorra.js`.

8.2 Registro de pisos y offsets

```
// js/pisos/registro.js
export const PISOS = [
  { id: 'piso1', num: 1, modulo: () => import('./piso1.js'), contenido: () => import('../contenido/p
  { id: 'piso2', num: 2, modulo: () => import('./piso2.js'), contenido: () => import('../contenido/p
  // …
];
export const origenPiso = (num) => new THREE.Vector3(0, 0, -700 * (num - 1)); // piso1 sigue en el o
```

- Los pisos se separan **700 m en el eje Z**; la casa y el exterior siguen en X = ∓400. La niebla y `camera.far` (120 m en interiores) garantizan que nunca se vea otra zona.

- **Carga diferida:** `irA('piso3')` hace `await PISOS[n].modulo()`, construye el piso con sus piezas (tras el fundido a negro) y lo guarda en caché. Se mantienen construidos como máximo el piso actual, el anterior y el siguiente; el resto se liberan (`dispose` de geometrías y materiales **propios**; las geometrías de KayKit son compartidas entre clones: llevar un contador de uso o no liberarlas).
- Las piezas se cargan por carpeta (`cargarPiezas`) y se cachean en un `Map` global por nombre, para no descargar dos veces `wall.glb`.

8.3 Contrato de un piso

```
// js/pisos/pisoN.js
export async function construir(ctx) {
  // ctx: { escena, origen, fx, piezas, contenido, paletas, mecanicas, sonido }
  const sala = await crearSalaDeTorre(ctx, { ancho: 6, largo: 9, filasMuro: 2, ventanas: [...], esta
  const desafios = [
    ctx.mecanicas.elegir.crear(ctx, contenido.soportes, { pos: [7, 3], sello: 'soportes' }),
    ctx.mecanicas.ordenar.crear(ctx, contenido.unidades, { pos: [-7, 4], sello: 'unidades' }),
    ctx.mecanicas.clasificar.crear(ctx, contenido.copias321, { pos: [0, -8], sello: 'copias' }),
  ];
  return {
    zona: sala.zona,                 // cumple la interfaz de zona (sección 2.3)
    sellos: ['soportes', 'unidades', 'copias'],
    desafios,
    update(dt, t, jugador) { sala.update(dt, t); desafios.forEach((d) => d.update?.(dt, t, jugador))
  };
}
```

`crearSalaDeTorre` generaliza `mazmorra.js`: recibe dimensiones en baldosas de 4 m y listas de elementos decorativos, coloca suelo, dos filas de muros, columnas con antorchas (máx. 6 luces), puerta norte con hoja giratoria y **3 runas** (una por sello), escalera y entrada sur, y devuelve la zona con `limitar`, `suelo` y `limitarCamara` ya resueltos (con la lógica de puerta/escalera que hoy vive en `world.zona`).

8.4 Estado por piso

```
state.pisos = {
  piso2: { sellos: { soportes: false, unidades: false, copias: false }, puertaAbierta: false, datos:
};
state.criterios = { '1.1': { aciertos: 0, fallos: 0 }, … };   // informe (sección 14)
```

`breakSeal(clave)` pasa a `romperSello(pisoId, clave)`: enciende la runa correspondiente, suena, suma saber, comprueba si están los tres y abre la puerta del piso.

8.5 Acciones genéricas

`claveAccion/valido/aplicarEvento` se sustituyen por un **registro de manejadores**:

```
registrarAccion('mec', {
  clave: (d) => d.unica ? `mec:${d.piso}:${d.mec}:${d.paso}` : null,
  valido: (d) => mecanica(d).validar?.(d) ?? true,
  aplicar: (d, soyAutor) => mecanica(d).aplicar(d, soyAutor),
});
```

Cada mecánica decide qué pasos son únicos (romper su sello) y cuáles repetibles (un cristal equivocado). Los tipos existentes (`cristal`, `salir`, `torre`, `leccion`, `sello`, `comentario`, `fin`) se registran igual. Añadir `irPiso` (el anfitrión decide el piso en el Portal de los pisos).

8.6 Contenido por piso y banco de preguntas

```
// js/contenido/piso2.js
export default {
  id: 'piso2', nombre: 'Piso II · La Bóveda de la Memoria', bloque: 'I', criterios: ['1.1'],
  intro: [...], sellosRotos: [...], memoria: '…fragmento de historia…',
  lecciones: { almacenamiento: { titulo, resumen, paginas, despues } , … },
  soportes: { … datos de la mecánica elegir … },
```

```
  preguntas: [ { id: 'p2-01', concepto: 'almacenamiento', criterio: '1.1', tipo: 'opcion', texto, op
  pistas: { soportes: '…', … },
};
```

`contenido/preguntas.js` reúne las de todos los pisos. La varita elige entre las de conceptos **aprendidos** (de cualquier piso), priorizando las falladas (repaso espaciado sencillo: peso ×3 a las falladas, ×0,5 a las acertadas dos veces seguidas).

8.7 Tipos de pregunta (ampliar `ui.quiz`)

`tipo`	Interfaz	Corrección
`opcion`	la actual (1-4)	índice
`vf`	Verdadero / Falso	booleano
`numero`	campo numérico con unidad	`Math.abs(r - esperado) <= tolerancia`
`texto`	campo corto	lista de respuestas aceptadas tras normalizar (minúsculas, sin tildes ni espacios sobrantes)
`orden`	reordenar 3-6 elementos (arrastrar o teclas)	igualdad de secuencia
`multiple`	varias casillas	conjunto exacto

Todas conservan `explicacion` y llaman a `onRespuesta(ok)`. Las de tipo `numero` son esenciales en los pisos II y III (conversiones y pesos de archivos).

8.8 El Portal de los pisos

Un atril en la casa de Aldric (o una opción de la portada, "Repasar un piso") que lista los pisos desbloqueados. Parámetro de URL `?docente=1` desbloquea todos (para repasar la extraordinaria). En equipo solo el anfitrión elige (`accion('irPiso', {piso})`). Al entrar así en un piso, las lecciones de pisos anteriores no se exigen, pero la varita solo pregunta lo aprendido en la sesión.

8.9 Progreso guardado

`localStorage.torreMorvath = { version: 2, pisos: { piso1: { completado, mejorPrecision, segundos } }, criterios: {...} }`, con migración desde la versión 1 actual. Siempre dentro de `try/catch` (navegadores sin almacenamiento).

---

9. Catálogo de mecánicas reutilizables

Cada mecánica es un módulo en `js/mecanicas/` que exporta `crear(ctx, def, opciones)` y devuelve `{ interactuables[], update?, aplicar(d, soyAutor), validar?(d), estado }`. Los interactuables tienen la misma forma que hoy (`{ zona, pos, r, enabled(), prompt(), action() }`). **Regla de red:** `action()` nunca cambia el estado compartido directamente; llama a `accion('mec', {piso, mec, paso, …})` y el cambio real ocurre en `aplicar()`, que se ejecuta en todos los jugadores.

9.1 `elegir` (generaliza el altar del Piso I)

Objetos sobre pedestales (`column` de Dungeon) con cristal o pieza representativa y etiqueta `makeLabel`. Uno o varios correctos; explicación por opción (`porque`). Opciones: `enunciado` (la lección previa), `modo: 'uno' | 'varios'`, `encargos[]` para rondas sucesivas (Piso II: cada encargo exige un soporte distinto; hay que acertar 3 encargos seguidos para el sello). Paso único: `sello`; paso repetible: `fallo`.

9.2 `clasificar` (llevar objetos a su sitio)

El jugador **coge** un objeto con E (flota sobre su cabeza, se sincroniza con `pos` añadiendo `lleva: idObjeto`) y lo **deja** con E en un receptáculo (estantería, cofre, pedestal). Validación al soltar: si es correcto se queda y brilla; si no, vuelve a su sitio con la explicación. Sello al completar todos. En equipo, varios pueden llevar objetos distintos a la vez (el anfitrión valida cada entrega: `paso: 'entregar', objeto, destino`). Usado en: III (formatos), VII (licencias), XIII (las cinco V).

9.3 `ordenar` (secuencias)

Losas o placas en el suelo numeradas por posición; el jugador coloca fichas (con `clasificar`) o, en la versión simple, una interfaz de reordenar (`ui` tipo `orden`). Usado en: II (unidades de menor a mayor), VII (fases de curación), XI (diagrama de flujo: aquí las losas tienen forma de símbolo y además se valida la **estructura**, no solo el orden: un rombo de decisión con dos salidas).

9.4 `palancas` (bits)

Fila de N palancas (8 por defecto) modeladas con piezas de Dungeon (`barrier_column` + una manivela procedural) o con cristales que se encienden. Cada palanca es un bit (valores 128…1 en runas encima). Un pedestal muestra el objetivo (decimal, hexadecimal o negativo). Estado compartido: el vector de bits (`paso: 'bit', i, v`), validado por el anfitrión; cualquier jugador puede tocar cualquier palanca. Complemento a 2: la puerta roja pide p. ej. –5 → `11111011`; la lección enseña "invertir y sumar 1". Mostrar en vivo el valor decimal (sin signo y con signo) para que se aprenda experimentando.

9.5 `conectar` (cables y topologías)

Nodos (islas, dispositivos) con un punto de anclaje. El jugador pulsa E en un nodo (se "engancha" un cable que sigue su mano: `THREE.TubeGeometry` sobre una `CatmullRomCurve3` que se regenera cada fotograma, material emisivo) y E en otro para fijarlo. Grafo en `estado.aristas`. Validadores reutilizables:

- `conexo(grafo)` (BFS);
- `estrella(grafo, centro)` (todos los nodos de tipo equipo tienen grado 1 y conectan al switch);
- `sinPuntoUnico(grafo)` (no hay puentes: algoritmo de Tarjan) para enseñar la malla;
- `rutaExiste(a, b)`.

Tipos de cable (UTP, fibra, coaxial, inalámbrico) como propiedad de la arista; algunos enlaces exigen un tipo (el cable submarino entre islas, fibra). En equipo, cada jugador puede tender cables a la vez.

9.6 `terminal`

Superposición HTML tipo consola (fuente monoespaciada, fondo negro, texto verde apagado con estilo mágico) con un **intérprete de comandos simulado** alimentado por el estado de `conectar` y `formulario`:

- `ipconfig` / `ipconfig /all`: IP, máscara, puerta de enlace, DNS y MAC de la isla actual.
- `ping <destino>`: 4 respuestas con tiempos plausibles si hay ruta; "Tiempo de espera agotado" si no; "no se pudo encontrar el host" si el DNS está mal.
- `tracert <destino>`: lista de saltos siguiendo el camino BFS; se corta en el enlace roto.
- `help`, `cls`; historial con flechas.

El sello se da al **diagnosticar**: un comando `reparar <enlace>` o una pregunta final sobre dónde estaba el fallo. La terminal es individual (cada uno la usa en su pantalla); solo la reparación es acción compartida.

9.7 `formulario` (configuración de dispositivos)

Panel con campos validados: IP (IPv4 con regex y rango), máscara (CIDR o decimal), puerta de enlace, DNS, SSID, tipo de cifrado (desplegable: abierto, WEP, WPA2, WPA3) y contraseña (reutilizar el medidor de robustez del Piso I). Validación semántica: `mismaSubred(ip, puerta, mascara)`, IP privada (10/8, 172.16/12, 192.168/16), no repetir IPs, no usar la dirección de red ni la de difusión. Mensajes de error pedagógicos, no genéricos.

9.8 `editorWeb`

Superposición con dos editores (`<textarea>` con numeración de líneas; opcionalmente CodeMirror 6 en el futuro) para HTML y CSS, y una **vista previa en `<iframe sandbox>` sin `allow-scripts`** alimentada por `srcdoc` (seguridad: el código del alumno nunca se ejecuta, y el iframe no comparte origen). Botones de ancho 375 / 768 / 1280 px para el sello *responsive*. **Validación sin ejecutar código:**

- HTML: `new DOMParser().parseFromString(html, 'text/html')` y comprobar requisitos del reto (`h1` único, `img[alt]` no vacío, `ul > li` ≥ 3, `a[href]`, `link rel=stylesheet` o `style`, `html[lang]`).

- CSS: `const hoja = new CSSStyleSheet(); hoja.replaceSync(css)` y recorrer `cssRules` para comprobar selectores y propiedades (color, `font-family`, márgenes, una `@media`).
- Contraste: calcular la ratio WCAG con los colores declarados de texto y fondo (fórmula de luminancia relativa); ≥ 4,5:1 pasa.

La auditoría del "heraldo ciego" es un conjunto de estas comprobaciones con mensajes en el tono del juego. **Todo local, nada se publica en internet** (el paso "publicar" se simula con el mapa dominio → hosting → servidor → URL).

9.9 `modelado` (forja 3D)

- Espacio de trabajo: yunque con rejilla y ejes X (rojo), Y (verde), Z (azul) visibles.
- Primitivas `BoxGeometry`, `SphereGeometry`, `CylinderGeometry` (material mate).
- Manipulación con `TransformControls` (three/addons, incluido en la versión 0.170; modos mover, rotar, escalar con las teclas G, R, S). Cuando la forja está activa, el control de cámara normal se desactiva.
- **Operaciones booleanas** con `three-bvh-csg@0.0.17` + `three-mesh-bvh@0.8.3` (ver sección 11: la 0.0.18 exige three ≥ 0.179 y **rompería** el juego).
- **Comprobar la llave:** comparar el sólido del alumno con el molde por **muestreo volumétrico**: rejilla de 24^3 puntos en la caja común; un punto está "dentro" si un rayo desde él corta la malla un número impar de veces (`three-mesh-bvh` acelera los raycasts). Se calcula la intersección sobre la unión (IoU): ≥ 0,85 abre la puerta; se muestra el porcentaje como pista.
- Estadísticas en vivo (vértices, aristas, caras) para el sello 2.
- **Exportar:** `STLExporter` (three/addons) + `Blob` + enlace de descarga iniciado por el usuario.
- **Red:** la forja es individual (cada uno forja su llave); solo "entregar la llave" es compartido. Variante cooperativa opcional: el equipo comparte una pieza y el anfitrión sincroniza las transformaciones a 10 Hz.

9.10 `enemigos` (bestiario del malware)

- **Modelos:** pack **KayKit Skeletons** (CC0, requiere aprobación de descarga: `github.com/KayKit-Game-Assets/KayKit-Character-Pack-Skeletons-1.0`; ~20 MB de repositorio, versión gratuita con 4 esqueletos animados) + animaciones compatibles del pack Adventurers (esqueleto de huesos común). Paleta nueva con el perfil `personaje`.
- **IA:** máquina de estados `patrulla → persigue → actua → desterrado`, navegación por puntos de ruta (sin navmesh; salas simples) y evitación de columnas con los `colliders`.
- **Comportamientos por tipo:**
  - `virus`: dormido dentro de un cofre hasta que alguien lo abre.
  - `gusano`: se duplica cada N segundos por los pasillos (tope de 6).
  - `troyano`: parece un cofre dorado hasta que te acercas.
  - `ransomware`: se sienta sobre un cofre, lo encadena y exige oro.
  - `spyware`/`keylogger`: sigue al jugador a distancia y muestra en un bocadillo "lo que ha copiado".
  - `botnet`: esqueletos que siguen a un nigromante.
- **Desterrar:** apuntar con la varita (generalizar `pickTarget` a cualquier objeto con la propiedad `diana`) → pregunta "¿qué tipo es?" → contramedida correcta (antivirus, cortafuegos, restaurar copia, actualizar, no pagar).
- **Nada violento:** los enemigos no dañan; roban "saber" o empujan al jugador a la entrada de la sala.
- **Red:** el **anfitrión simula** a los enemigos y envía `ene` a 8 Hz (posición, estado y animación de cada uno); los invitados solo interpolan y envían acciones (`mec` con `paso: 'desterrar', id`).

9.11 `casos` (juicios y conversaciones)

Árboles de decisión definidos en el contenido: nodos con texto, orador (NPC con modelo KayKit o una

silueta) y opciones; cada opción lleva a otro nodo, suma o resta "confianza", y los nodos finales explican. Interfaz: extender `ui` con `eleccion(texto, opciones) → Promise<indice>` (una caja de diálogo con botones). La **simulación de chat** del Piso X reutiliza el estilo del panel de chat, pero con mensajes guionizados de NPCs (nunca contacto real). En equipo, el anfitrión muestra el caso a todos y **vota** la opción (mayoría, empate = anfitrión): `paso: 'voto'`.

9.12 `cofreCripto` (criptografía asimétrica real)

WebCrypto (`crypto.subtle`, nativo, sin dependencias): cada jugador genera un par RSA-OAEP 2048 al entrar en el piso; la clave pública se comparte por la red (`paso: 'clavePublica'`, en formato `spki` base64). Para abrir el cofre de otro, un jugador cifra la "runa secreta" con la clave pública del destinatario; solo el navegador del destinatario puede descifrarla con su privada (que nunca sale de su ordenador) y así abrir el cofre. Se enseña mostrando los textos cifrados de verdad. Versión cifrado simétrico: AES-GCM con una contraseña compartida (PBKDF2). **Solo:** Aldric hace de segundo jugador (su par se genera localmente).

9.13 `bloques` (programación visual)

Editor de bloques propio y ligero (DOM con arrastrar y soltar; no Blockly, que pesa ~1 MB):

- Bloques disponibles: `avanzar`, `girar izq/der`, `si hay muro … si no …`, `repetir N`, `mientras no llegues`, variables, `+ - * /` y comparaciones.
- El programa se guarda como árbol (JSON) y se **genera JavaScript ES5** mostrado al lado, junto al pseudocódigo en español.
- Se ejecuta **paso a paso** con **JS-Interpreter** (`js-interpreter@6.0.2`, Apache-2.0, `lib/js-interpreter.min.js`, 94 KB, UMD → cargar con `<script>`), que permite ejecutar una instrucción por fotograma, resaltar el bloque activo y **cortar bucles infinitos** con un presupuesto de pasos.
- API expuesta al intérprete: `avanzar()`, `girar(dir)`, `hayMuro()`, `enMeta()`, `decir(texto)`.
- El autómata es un personaje KayKit (el Bárbaro o un esqueleto) que se mueve por una cuadrícula de 2 m.

9.14 `codigo` (programación textual y POO)

JS-Interpreter solo entiende **ES5** (sin `class`), así que para la POO se usa otro enfoque: el código del alumno se ejecuta en un **Web Worker creado desde un Blob**:

- Motor real del navegador (ES2023, con clases y herencia).
- Aislado del DOM y del juego.
- Se cierra con `worker.terminate()` si pasa de 2 s.
- El Worker recibe una API mínima (`crearCriatura(obj)`, `leerPergamino(nombre)`, `escribirPergamino(nombre, texto)`, `mostrar(matriz5x5)`) implementada con `postMessage`; el juego valida y materializa los resultados (aparecen criaturas en la sala, se enciende la placa…).
- Editor: `textarea` con numeración y resaltado mínimo (o CodeMirror 6 si el usuario aprueba la dependencia).
- Errores de sintaxis capturados y explicados; la comprobación de "comentado" cuenta los comentarios `//` y `/ /` por función.

9.15 `placa` (micro:bit/Arduino simulada)

Objeto 3D: placa verde con una matriz de 5×5 LED (esferas emisivas) y dos botones A y B. Se programa con `codigo` o con `bloques` (eventos `alPulsarA`, `alPulsarB`, `mostrar(patron)`, `pausa(ms)`). La puerta se abre si muestra el símbolo pedido tras pulsar A.

9.16 `sql` (archivo de Umbravel)

- **sql.js** (SQLite compilado a WASM; MIT; `sql-wasm.js` + `sql-wasm.wasm` de 658 KB). Se carga solo al entrar en el Piso XIII.
- La base se crea desde un guion SQL del contenido: tablas `prisioneros(id PK, nombre,`

`celda_id FK)`, `celdas(id PK, planta, ala)` y `guardias(id PK, nombre, celda_id FK)`, con datos de la historia.
- Consola con historial y resultados en tabla.
- **Retos:** `SELECT` con `WHERE`, `JOIN` y un `UPDATE` para "liberar" a un prisionero.
- Se permite cualquier SQL porque la base vive en memoria y se reconstruye al salir.
- Alternativa más ligera: alasql (JS puro, 516 KB), menos fiel a SQL estándar.

9.17 `oraculo` (aprendizaje automático sin librerías)

- **Supervisado:** extractor de rasgos sobre los mensajes de los pergaminos del Piso I (¿mete prisa?, ¿pide contraseña?, ¿enlace raro?, ¿premio?, ¿remitente oficial?) → vector binario; clasificador **k-NN** o **perceptrón** escrito a mano (unas 60 líneas). El alumno etiqueta ejemplos arrastrándolos a dos cestas; el oráculo "entrena" (animación) y clasifica pergaminos nuevos. Si solo se le dan ejemplos de un tipo, falla: **sesgo** visible.
- **No supervisado:** k-means sobre colores de cristales (se ven los grupos formarse).
- **Por refuerzo:** Q-learning en una cuadrícula de 5×5 con el autómata del Piso XI (se ve cómo mejora episodio a episodio).
- **Alucinación:** el oráculo "generativo" responde con plantillas que a veces inventan datos; el reto es detectar cuáles.

TensorFlow.js no es necesario (evita más de 1 MB de dependencia).

9.18 `realidad` (RV/RA en la cima)

- **Editor de escena en el juego:** colocar 3-5 objetos (piezas KayKit), asignar a cada uno un activador (marcador de imagen, QR o geolocalización, simulados en el juego) y una interacción (al mirar, al tocar o al acercarse). El reto es que la escena "revele" los puntos débiles de Morvath.
- **Modo real opcional:** `VRButton`/`ARButton` de three/addons (WebXR). Requiere **HTTPS** (GitHub Pages) y un dispositivo compatible (Chrome en Android con ARCore o el navegador de un visor Meta Quest). Si `navigator.xr` no existe, el botón no aparece.
- **Exportación opcional** de la escena a un HTML de **A-Frame** (MIT, por CDN) que el alumno se descarga y abre en el móvil.
- Nada de cámara ni geolocalización reales dentro del juego (privacidad de menores).

---

10. Especificación técnica piso a piso

Formato de cada ficha:

- **Sala:** baldosas de 4 m.
- **Piezas:** además de las comunes.
- **Mecánicas.**
- **Contenido nuevo.**
- **Red.**
- **Pruebas mínimas.**

Todas las salas usan `crearSalaDeTorre` (sección 8.3) y las piezas comunes de Dungeon Remastered ya presentes: `wall`, `wall_cracked`, `wall_arched`, `wall_archedwindow_gated`, `wall_pillar`, `wall_shelves`, `wall_doorway`, `floor_tile_large`, `floor_tile_large_rocks`, `floor_tile_big_grate`, `pillar`, `column`, `torch_mounted`, `stairs_walled`, estandartes y atrezo. Las piezas adicionales se copian desde `assets/_descargas/KayKit-Dungeon-Remastered-1.0-main/…/gltf/` a `assets/modelos/mazmorra/` (el pack completo ya está descargado; no requiere aprobación).

10.1 Piso I · La Cámara de los Sellos (migración y ajuste)

- **Migración:** reproducir la sala actual con `crearSalaDeTorre` (6 × 9, dos filas de muros) y las mecánicas `quiz` (cartel), `elegir` (altar) y `dianas` (pergaminos + varita).

- **Contenido:** subir el nivel a Bachillerato. Preguntas de MFA, gestor de contraseñas, smishing, vishing e ingeniería social, todas etiquetadas con `criterio: '3.1'` o `'3.3'`. Un quinto pergamino de *smishing* (SMS de paquetería).
- **Pruebas:** la batería actual (solo y equipo de 3) debe pasar sin cambios.

10.2 Piso II · La Bóveda de la Memoria (I.1, crit. 1.1)

- **Sala:** 6 × 9; ambiente de cámara acorazada; en el centro, un cofre gigante (`chest_gold` escalado ×3).
- **Piezas extra:** `chest_gold`, `trunk_large_B/C`, `shelf_large`, `shelves`, `keyring_hanging`, `coin_stack_*`.
- **Sello 1 · `elegir` con encargos:** 9 soportes representados por objetos-símbolo. El HDD, un disco de piedra con cabezal; el SSD, un cristal plano; la cinta, un pergamino enrollado; el Blu-ray, un disco reflectante; el pendrive y la SD, amuletos; el NAS, un cofre con antena; la nube, una nube de partículas. Cada encargo tiene uno o dos soportes válidos y una explicación por opción; hay que acertar 3 encargos seguidos.
- **Sello 2 · `ordenar` + preguntas `numero`:** placas bit → PB. Luego 3 conversiones: "¿cuántos MB son 3,5 GB?", "¿cuántos GiB muestra Windows en un disco de 1 TB?" (931,3 con tolerancia 0,5) y "¿cuántos bytes hay en 2 KiB?".
- **Sello 3 · `clasificar` (regla 3-2-1):** tres "copias" (orbes) que hay que llevar a dos cofres de tipo distinto y a un portal de "fuera de la torre". Después, `quiz` sobre copia completa, incremental y diferencial.
- **Opcionales:** `elegir` de sistema de archivos (4 casos) y una lección de cifrado de unidad (BitLocker, FileVault, LUKS).
- **Lecciones:** `almacenamiento`, `unidades`, `copias`, `sistemasArchivos`.
- **Red:** todos los pasos son `mec`; en `clasificar` pueden llevar orbes distintos a la vez.
- **Pruebas:** completar los 3 sellos solo; en equipo, dos jugadores entregando orbes simultáneamente sin duplicar.

10.3 Piso III · El Scriptorium Binario (I.2, crit. 1.1)

- **Sala:** 6 × 9 con escritorios de escriba (`table_long_decorated_A`, `candle_*`, `bottle_*`, `shelf_small_candles`) y pergaminos flotantes.
- **Sello 1 · `palancas`:** 3 puertas en serie.
  - Puerta 1: decimal → binario (p. ej. 77).
  - Puerta 2: hexadecimal → binario (0x3C).
  - Puerta 3, roja: –5 en complemento a 2.
  - En pantalla se ven el valor sin signo, con signo y en hexadecimal.
- **Sello 2 · inscripción `texto`:** secuencias de códigos (`72 111 108 97` → "Hola"; UTF-8 de "ñ" = `C3 B1`) con una tabla ASCII en un atril y una runa-emoji que exige Unicode.
- **Sello 3 · `numero` + `clasificar`:**
  - Pesos: imagen 1920×1080×24 bits = 6.220.800 B ≈ 5,93 MiB; audio de 3 min a 44,1 kHz, 16 bits, estéreo = 31.752.000 B.
  - Clasificar formatos (con/sin pérdida; códec frente a contenedor) en dos estanterías.
- **Guiño técnico:** botón "ver malla" que pone el modelo del jugador en modo `wireframe` y muestra su número de vértices y caras (`geometry.attributes.position.count`, `index.count/3`).
- **Pruebas:** comprobar las conversiones con una función de referencia (`(n >>> 0).toString(2)`, `(256 + n) % 256` para complemento a 2 en 8 bits).

10.4 Piso IV · Los Puentes Flotantes (I.3, crit. 1.2)

- **Sala:** no es una mazmorra, es **exterior nocturno**: 7 islas flotantes (baldosas `hex_*` de Medieval Hexagon escaladas ×4 sobre columnas de roca) separadas por el vacío, con niebla. La zona

cambia `suelo(p)`: si el jugador está fuera de una isla y no hay puente, cae y reaparece en la última isla (sin daño).

- **Puentes:** cuando se conecta un enlace, aparece una pasarela de luz entre dos islas (`floor_wood_small` repetido con material emisivo).
- **Piezas extra (ya descargadas):** Medieval Hexagon `hex_grass`, `hex_coast_*`, `building_tower_A` (faro), `building_well`, `building_windmill`; Dungeon `barrier*`.
- **Sello 1 · `conectar`:**
  - Cada isla lleva un dispositivo (router en la central, switch, punto de acceso y equipos en las demás).
  - Hay que montar una **estrella** en el "barrio" de 4 islas; después, conectar las dos islas "Tenerife" y "Gran Canaria" con **fibra** (cable submarino).
  - Por último, **añadir redundancia** para que ningún enlace sea imprescindible (`sinPuntoUnico`).
- **Sello 2 · `formulario`:** configurar IP, máscara, puerta de enlace y DNS de 3 equipos, en la misma subred que el router (192.168.20.0/24). La lección explica DHCP; un botón "activar DHCP" asigna automáticamente y se comparan los dos métodos.
- **Sello 3 · `terminal`:** Morvath rompe un enlace; con `ping` y `tracert` se localiza el enlace roto y se repara.
- **Opcionales:**
  - Faro Wi-Fi: `formulario` con SSID, WPA3 y contraseña robusta.
  - Chatarrería RAEE: `clasificar` en reutilizar, reciclar o reparar, con una lección sobre obsolescencia programada.
- **Guiño:** una lección explica que el modo equipo del juego es P2P en estrella (enlazar con `red.js`).
- **Red:** los cables son `mec` (un `paso: 'cable'` con origen, destino y tipo; `paso: 'quitar'`); la terminal es individual.
- **Pruebas:** validadores de grafo con casos conocidos (`node --test` sobre un módulo puro `grafo.js`).

10.5 Piso V · El Taller de los Heraldos (II base, II.1.1, II.2.4; crit. 2.2, 2.3)

- **Sala:** 6 × 9 tipo imprenta/taller (`table_medium*`, `shelves`, `banner_*` como carteles en blanco).
- **Lección inicial:** PLE, trabajo colaborativo, historial de versiones y lenguaje inclusivo. El historial se ilustra con el propio `git log` del juego como ejemplo.
- **Sello 1 · `editorWeb` (cartel "Se busca a Aldric"):**
  - Requisitos: `h1`, `p`, `img` con `alt`, `ul` con 3 `li`, un enlace y un CSS con al menos 3 propiedades.
  - Imágenes disponibles: 3 PNG del propio juego (capturas pequeñas en `assets/web/`), referenciadas por ruta relativa. Se inyectan como `data:` URL en el `srcdoc`, porque el iframe con sandbox no tiene origen.
- **Sello 2 · `editorWeb` responsive:** el cartel debe verse bien a 375 px. Validación: existe una regla `@media (max-width: …)` y la imagen no desborda (`max-width: 100%`).
- **Sello 3 · auditoría:** se entrega un cartel "de Morvath" con 5 fallos de accesibilidad (sin `alt`, contraste 2:1, sin `lang`, títulos saltados h1→h4, texto "haz clic aquí") y hay que corregirlos. Cada corrección es detectada por las comprobaciones de la sección 9.8.
- **Opcional:** `ordenar` el camino de publicación (dominio → DNS → hosting/servidor → URL) y `elegir` un CMS según el caso.
- **Red:** cada jugador edita su propio cartel; el sello se rompe cuando el **primero** lo consigue, y el cartel ganador aparece colgado en la sala para todos. Se envían el título y la imagen elegidos y cada cliente dibuja la textura con canvas. No se usa `html2canvas`, que es una dependencia pesada.
- **Seguridad:** nunca `allow-scripts` en el iframe; nunca `innerHTML` del HTML del alumno en el documento del juego.

10.6 Piso VI · La Forja de las Formas (II.1.2, II.1.3; crit. 2.1)

- **Sala:** 6 × 9 tipo fragua. El yunque en el centro y un horno (luz naranja parpadeante, reutilizar las antorchas). Una cerradura gigante en la puerta con el molde de la llave como hueco.
- **Sello 1 · `modelado`:** llave objetivo = cilindro (caña) + cubo (paletón) – cilindro pequeño (ojo) ∪ 2 dientes (cubos). IoU ≥ 0,85.
- **Sello 2:** panel de estadísticas de la malla y preguntas `numero`/`opcion` sobre vértices, aristas y caras, STL frente a OBJ y el laminado. Botón "descargar STL".
- **Sello 3 · `elegir` con encargos:** "una maqueta de casa para arquitectura" → SketchUp; "un personaje animado" → Blender; "una pieza mecánica con medidas exactas" → FreeCAD; "primer diseño en 1.º de ESO" → Tinkercad.
- **Opcional:** visor de realidades. Tres ventanas: RV (inmersión total), RA (objetos sobre el mundo real) y RM (interacción entre ambos), con ejemplos.
- **Dependencias:** `three-bvh-csg@0.0.17` y `three-mesh-bvh@0.8.3` en el importmap (sección 11), cargadas solo en este piso con `import()`.
- **Red:** individual; "entregar la llave" es compartido.
- **Pruebas:** IoU de la llave de referencia consigo misma = 1; de un cubo suelto < 0,5.

10.7 Piso VII · La Gran Biblioteca (II.2.1-II.2.3; crit. 2.3, 3.3)

- **Sala:** 8 × 9 (más grande), con muros `wall_shelves` en toda la planta baja y `shelves`/`shelf_large` exentas formando pasillos. Iluminación de velas.
- **Sello 1 · `clasificar` licencias:** 10 obras (libros con portada generada por canvas: título, autor y un icono de licencia **solo en la lección**, no en el libro) que hay que llevar a 7 estanterías: copyright, dominio público, CC BY, CC BY-SA, CC BY-NC, CC BY-ND y software libre/propietario. Cada libro describe su caso ("puedes copiarlo y venderlo si citas al autor" → BY).
- **Sello 2 · tablón de pregones (`casos` + herramientas):** 5 noticias. Herramientas: lupa (búsqueda inversa: muestra que la foto es de otro año), sello de fecha, ficha del autor y verificador. Por noticia hay que elegir la etiqueta (bulo, verdadera, sátira, clickbait o deepfake) y la acción (no difundir, reportar, contrastar).
- **Sello 3 · buscador:** interfaz de buscador falso con índice local de unos 40 "libros". El alumno escribe una consulta que se **evalúa de verdad** con un parser propio (frases entre comillas, `site:`, `filetype:`, `-palabra`) sobre el índice. Reto: que el libro buscado salga el primero. Después, `ordenar` las 5 fases de la curación.
- **Opcional:** la burbuja de filtros. Un "algoritmo" que recomienda según lo que el jugador ha pulsado en el tablón, y se ve cómo se estrecha.
  - **Hecho** (2026-10-01) como el **espejo de las recomendaciones** (junto a la entrada, al este), individual: 5 muros de 6 titulares de 6 temas; cada elección da peso a su tema (`js/mecanicas/burbuja.js`, 1 + 4·clics², pruebas en `burbuja.test.mjs`). Al final compara los temas del primer muro y del último, enseña la lección `burbuja` (burbuja de filtros y cámara de eco) y hace una pregunta; +20 de saber.
- **Pruebas:** el parser de búsqueda como módulo puro con pruebas en Node.

10.8 Piso VIII · Las Criptas del Contagio (III.1.1, III.1.2; crit. 3.1)

- **Sala:** 6 × 12 en forma de L, con celdas (`wall_gated`, `barrier*`), cofres y tumbas (Halloween Bits: `coffin*`, `grave_*`).
- **Dependencia de recursos:** pack **KayKit Skeletons** (pedir aprobación). Sin él, los enemigos usan el esqueleto del Bárbaro con paleta verde fantasmal (plan B).
- **Sello 1 · `enemigos`:** 6 criaturas, una de cada tipo, que se destierran identificándolas.
- **Sello 2 · muralla cortafuegos (`formulario` de reglas):** tabla de tráfico entrante (origen, puerto y servicio) con permitir o denegar. Reto: dejar pasar HTTPS (443) y bloquear el acceso remoto (3389) y un puerto sospechoso.
- **Sello 3 · la tríada (`elegir` ×3):** para cada amenaza (espía, alterador y saboteador), la medida que

protege la confidencialidad (cifrado), la integridad (hash/firma) o la disponibilidad (copias y redundancia).

- **Opcionales:** pozo de la Wi-Fi pública (casos) y taller del parche (actualizar cuatro "dispositivos" en orden).
- **Red:** enemigos con autoridad del anfitrión (sección 9.10, mensaje `ene`).
- **Pruebas:** en equipo de 3, desterrar el mismo enemigo a la vez: debe contar una sola vez.

10.9 Piso IX · El Tribunal de los Datos (III.2, III.3; crit. 3.2, 3.3)

- **Sala:** 6 × 9 en forma de tribunal: estrado (`table_long_tablecloth`), bancos (`bench`, `chair`) y estandartes con balanza.
- **NPCs:** personajes del pack Adventurers con otras paletas (Mago, Caballero), sin armas, con la animación `Idle`.
- **Sello 1 · `casos` (6 juicios):**
  - Una red social no borra las fotos de una alumna → supresión.
  - Un gimnasio vende los datos → oposición.
  - "Quiero llevarme mis datos a otra compañía" → portabilidad.
  - Una app pide consentimiento a una niña de 12 años → menor de 14.
  - Datos de salud → especialmente protegidos.
  - "¿Dónde reclamo?" → AEPD.
- **Sello 2 · `cofreCripto`:** en equipo con WebCrypto real; solo con Aldric. Después, lección sobre certificado digital, DNIe, Cl@ve, firma y sede electrónica; y un `ordenar` de los pasos de un trámite electrónico.
- **Sello 3 · espejo de la huella:**
  - Una "sombra" del jugador enumera su huella, generada a partir de las acciones de la partida (lo que escribió en el chat, su etiqueta, horas de juego). Es un ejemplo **real y local** de huella pasiva, sin enviar nada.
  - El reto: marcar qué es huella activa y qué pasiva, y configurar un panel de privacidad falso.
- **Pruebas:** WebCrypto disponible en `localhost` y HTTPS (es un contexto seguro); en `http://` con IP de red local **no** está disponible → comprobar `crypto.subtle` y degradar a una simulación.

10.10 Piso X · El Laberinto de las Sombras (III.4, III.5; crit. 3.3)

- **Sala:** laberinto con muros de 4 m (Dungeon) y niebla densa. Las "sombras" son siluetas oscuras (el shader del holograma en negro, reutilizando `hologram.js` con otro `uColor`).
- **Sello 1 · `casos` en formato chat simulado:** 4 conversaciones (smishing de paquetería, vishing del "banco", un desconocido que pide fotos y se hace pasar por alguien de tu edad (grooming), y una sextorsión con amenaza). Opciones de respuesta con consecuencias explicadas. Final común: guardar pruebas, no pagar, bloquear, contárselo a un adulto de confianza, **INCIBE 017** y denunciar.
- **Sello 2 · el testigo:** escena de ciberacoso en un grupo de chat simulado. Decisiones como testigo: no reenviar, apoyar a la víctima, reportar, avisar.
- **Sello 3 · reloj de arena:** preguntas y casos sobre bienestar digital, identidad positiva, netiqueta y violencias sexistas digitales.
- **Salvaguardas obligatorias:**
  - Textos revisados por un docente u orientador antes de publicarlos (dejar la bandera `revisadoDocente: false` en el contenido y un aviso en el modo docente).
  - Sin imágenes ni descripciones explícitas.
  - Un botón permanente "¿Necesitas ayuda? 017", visible en este piso.
- **Red:** los casos se votan en equipo.

10.11 Piso XI · El Taller de los Autómatas (IV.1, IV.2; crit. 4.1, 4.2)

- **Sala:** 6 × 9 con el suelo en **cuadrícula de 2 m** marcada (baldosas `floor_tile_small`, que miden 2×2) y trampas (`floor_tile_big_spikes`, `spikes`).
- **Sello 1 · `ordenar` estructural (diagrama de flujo):** losas con forma de símbolo. Valida que el grafo del diagrama sea equivalente al algoritmo pedido ("si la palanca está arriba, abre; si no, bájala y vuelve a comprobar").
- **Sello 2 · `bloques`:** 3 laberintos de dificultad creciente (secuencia; bucle; condicional dentro de un bucle). Se muestran JS y pseudocódigo.
- **Sello 3 · depuración:** un programa con un error de sintaxis (en la vista de código: falta un paréntesis y el intérprete da el error) y otro con un error de lógica (el autómata gira al lado contrario).
- **Red:** en equipo, cada uno programa un autómata; el sello 2 exige que todos lleguen; con variante individual si se juega solo.
- **Dependencia:** `js-interpreter@6.0.2` (pedir aprobación).

10.12 Piso XII · La Fragua del Código (IV.3; crit. 4.3, 4.4)

- **Sala:** 6 × 9 con mesas de trabajo, una placa gigante (5×5 LED) en la pared y moldes (`crates_stacked`) de criaturas.
- **Sello 1 · `codigo` POO:**
  - Plantilla con `class Criatura { constructor(nombre, color) {…} saludar() {…} }` y `class Esqueleto extends Criatura`.
  - Reto: crear 3 objetos, que aparecen en la sala con su nombre, y sobrescribir un método.
  - Todo se ejecuta en un Worker (sección 9.14).
- **Sello 2 · `placa`:** mostrar la runa de la puerta al pulsar A.
- **Sello 3 · libro de registro:** leer `pergamino_visitas.csv` (texto en memoria), contar visitas por día y escribir `resumen.txt`. Validación del contenido escrito y de que hay al menos un comentario por función.
- **Lección transversal:** tipos de lenguajes, compilado frente a interpretado (demostración: el mismo algoritmo en el editor de bloques y en JS), IDE, control de versiones y reutilizar código de otro jugador.
- **En equipo:** el anfitrión puede "compartir" su programa y los demás lo cargan y amplían.

10.13 Piso XIII · El Observatorio del Oráculo (V.1; crit. 4.3)

- **Sala:** circular, formada por muros en octógono; techo abierto con estrellas (reutilizar las de `exterior.js`) y un orbe gigante (el oráculo) en el centro.
- **Sello 1 · `oraculo`:** entrenamiento supervisado y demostración de sesgo; después, las variantes no supervisada y por refuerzo (visuales, sin sello).
- **Sello 2 · `sql`:** encontrar la celda de Aldric con `SELECT` + `JOIN` + `WHERE`; liberar con `UPDATE`.
- **Sello 3 · `clasificar`:** 10 ejemplos en las 5 V del big data; `casos` para detectar alucinaciones; lección de ética y del Reglamento Europeo de IA.
- **Dependencia:** `sql.js` (pedir aprobación).
- **Nota para el tono:** el propio juego se hizo con ayuda de IA; Aldric puede comentarlo con naturalidad al hablar de usos y límites.

10.14 Cima · El Trono de Morvath (V.2; crit. 4.5 + todos)

- **Sala:** azotea de la torre al aire libre, con la luna enorme (`exterior.js`), almenas (`wall_half`, `barrier`) y el trono (`building_castle` de Medieval Hexagon escalado como fondo).
- **Sello de la cima · `realidad`:** diseñar la escena de RA que revela 3 puntos débiles. Modo WebXR opcional.
- **Jefe final:** Morvath es el modelo Mage con paleta propia y el shader del holograma invertido. Tiene 5 fases, una por bloque:

- ○ Bloque I: le quita la energía a su escudo con una conversión binaria.
- ○ Bloque II: se detecta su bulo.
- ○ Bloque III: se destierra su malware.
- ○ Bloque IV: se corrige su algoritmo.
- ○ Bloque V: se reentrena su oráculo.

Cada fase reutiliza una mecánica ya construida en versión corta, con temporizador suave y sin castigo. En equipo, fases paralelas: cada jugador ataca un punto débil.

- **Final:** diálogo, créditos (con las licencias CC0 de KayKit) e informe por criterios.

---

11. Nuevas dependencias propuestas (verificadas el 30-09-2026)

Todas requieren la aprobación del usuario antes de añadirlas (enlace, licencia y tamaño).

Librería	Versión	Licencia	Tamaño	Para	Cómo cargarla
three-bvh-csg	**0.0.17**	MIT	83 KB	Booleanas del modelado (piso VI)	importmap `"three-bvh-csg": "https://cdn.jsdelivr.net/npm/three-bvh-csg@0.0.17/build/index.module.js"`
three-mesh-bvh	**0.8.3**	MIT	193 KB	Requisito de la anterior y raycasts rápidos	importmap `"three-mesh-bvh": "https://cdn.jsdelivr.net/npm/three-mesh-bvh@0.8.3/build/index.module.js"`
js-interpreter	6.0.2	Apache-2.0	94 KB	Ejecución paso a paso de bloques (piso XI)	`<script src="https://cdn.jsdelivr.net/npm/js-interpreter@6.0.2/lib/js-interpreter.min.js">` cargado bajo demanda
sql.js	1.14.2	MIT	~0,7 MB (WASM)	SQL real (piso XIII)	`initSqlJs({ locateFile: f => 'https://cdn.jsdelivr.net/npm/sql.js@1.14.2/dist/' + f })`
KayKit Skeletons	1.0	CC0	~8 MB (gratis)	Enemigos (piso VIII)	Descargar del GitHub oficial, como los otros packs

Incompatibilidad detectada: `three-bvh-csg@0.0.18` (la última) exige `three >= 0.179` y `three-mesh-bvh >= 0.9.7`. **Fijar 0.0.17 y 0.8.3** mientras el juego use Three 0.170. Si algún día se actualiza Three, actualizar ambas a la vez y revisar el `Hologram` (usa *chunks* de shader con nombres que cambian entre versiones), el `OutputPass` y el `UnrealBloomPass`.

Descartadas a propósito: Blockly (~1 MB), TensorFlow.js (>1 MB), html2canvas y CodeMirror (opcional más adelante). Todo lo que se puede escribir en menos de 200 líneas se escribe.

Nativas del navegador (sin dependencia): WebCrypto, WebXR, Web Workers, DOMParser, `CSSStyleSheet` constructable y Blob/URL para descargas.

---

12. Multijugador en los pisos nuevos

1. **Todo lo compartido pasa por `accion('mec', …)`** con autoridad del anfitrión (sección 8.5). Nada de cambiar el estado compartido en `action()`.
2. **Mensajes nuevos:**
   - `ene` (anfitrión → todos, 8 Hz): estado de los enemigos.
   - `pieza` (anfitrión → todos, 10 Hz): solo en la forja cooperativa opcional.
   - `snap` (anfitrión → invitado): instantánea del estado del piso al entrar en él (sellos, cables tendidos, objetos colocados). Motivo: un invitado lento puede llegar con eventos encolados; al entrar en un piso, el anfitrión envía el estado completo y el invitado lo aplica antes de procesar nada más.
3. **Presupuesto de tráfico:** con 5 jugadores, `pos` a 12 Hz son 60 mensajes/s de entrada en el anfitrión

y 240/s de salida (reenvíos). Es asumible, pero **no añadir más tráfico continuo**. Para los enemigos, empaquetar todos en un mensaje (`[{id, x, z, f, a, e}]`).

4. **Votos:** en `casos`, cada jugador envía `paso: 'voto'`. El anfitrión cierra la votación cuando han votado todos o a los 20 s, y anuncia el resultado.
5. **Retos por equipo:**
   - Cofre criptográfico: exige al menos 2 jugadores.
   - Autómatas simultáneos: cada jugador programa uno y deben llegar todos.
   - Cada reto de equipo lleva una variante para jugar solo (Aldric hace de compañero o se relaja la condición).
6. **Anfitrión caído a mitad de piso:** los invitados pasan a solitario con el estado que tenían. Como todo se aplica en todos los clientes, el estado es coherente, aunque los enemigos se congelan hasta que el invitado vuelve a simularlos. Implementar: si `red.activa` pasa a `false`, las mecánicas con autoridad (enemigos) pasan a simulación local.
7. **Contenido pesado individual** (terminal, editor web, forja, SQL): no se sincroniza. Solo los resultados.

---

13. Sonido de los pisos nuevos

`Sonido.crearCapa(nombre)` admite capas nuevas. Todas se construyen con las mismas primitivas (bordón, acordes, campanas, viento o ruido); cada zona declara `musica`.

Capa	Pisos	Carácter
`boveda`	II	Grave y metálico: bordón en re, campana lejana cada 15 s, eco largo
`scriptorium`	III, VII	Íntimo: caja de música en modo dórico, crepitar de velas
`archipielago`	IV	Abierto y ventoso: viento intenso, notas agudas espaciadas
`taller`	V, XI, XII	Rítmico suave: pulso de 90 bpm con ruido filtrado (martillo lejano)
`forja`	VI	Grave con golpes de yunque (ruido + tono corto) cada 2 s
`criptas`	VIII	Tensión: acordes disminuidos, susurros (ruido de banda estrecha modulado)
`tribunal`	IX	Solemne: acordes lentos en fa menor
`sombras`	X	Casi silencio: bordón muy grave y latidos
`observatorio`	XIII	Etéreo: pads agudos, reverberación máxima
`cima`	Cima	Épico: tema propio en 3 secciones que cambia en cada fase del jefe

Efectos nuevos:

- `palanca`: clic metálico con dos tonos.
- `cable`: zumbido que sube al conectar.
- `teclado`: clics de ruido corto, para la terminal y el editor.
- `enemigo`: gruñido de ruido filtrado con tono bajo.
- `desterrar`: campana invertida.
- `voto`: dos notas.

Mantener el volumen global por debajo del compresor actual y respetar la tecla M.

---

14. Evaluación, informe por criterios y privacidad

1. **Etiquetado:** toda pregunta, sello y caso lleva `criterio` (o una lista). La validación del contenido (sección 17) falla si falta.
2. **Registro:** `record(ok, criterio)` suma en `state.criterios[criterio]`. Los sellos cuentan como evidencia fuerte (peso 3); las preguntas de la varita, como débil (peso 1).
3. **Pantalla de fin de piso:** estadísticas actuales más una barra por criterio trabajado en el piso.

4. **Informe descargable** (botón "Informe para el profesor"):
    - Genera en el navegador un **CSV** (apodo, fecha, piso, criterio, aciertos, fallos, porcentaje, tiempo) y un **HTML imprimible**.
    - Se descargan con `Blob`.
    - **Nunca se envía a ningún servidor.**
5. **Privacidad (menores, normativa europea):**
    - Sin cuentas, sin correo y sin nombre real obligatorio (apodo).
    - Nada de analítica de terceros; nada de cámara ni micrófono.
    - El modo equipo solo transmite apodo, posición, acciones y chat, directamente entre navegadores; el broker de PeerJS solo ve el id de sala.
    - Documentarlo en una página "Privacidad" accesible desde la portada. Este mismo punto es contenido del Piso IX.

---

15. Accesibilidad, opciones y controles táctiles

Coherente con lo que se enseña en el Piso V (II.2.4):

- **Menú de opciones** (tecla O o engranaje en el HUD), guardado en `localStorage`:
    - ○ Volumen de música y efectos.
    - ○ Tamaño de texto (100 %, 125 %, 150 %, con variables CSS).
    - ○ Alto contraste.
    - ○ **Reducir movimiento:** sin sacudidas de cámara, sin parpadeo del holograma, menos partículas.
    - ○ Sensibilidad e inversión del ratón.
    - ○ Velocidad del texto de los diálogos.
    - ○ Calidad gráfica (alta o baja).
- **Teclado:** todo el juego debe poder jugarse sin ratón. Hoy la cámara solo gira arrastrando el ratón. Hay que añadir el giro con las teclas **J** y **L**, o con las flechas izquierda y derecha cuando se juega con WASD. No se pueden usar Q y E porque E ya sirve para interactuar.
- **Táctil (tabletas):**
    - ○ Joystick virtual a la izquierda y arrastrar a la derecha para la cámara.
    - ○ Botones en pantalla para E, F, G, H y el chat.
    - ○ Detectar con `matchMedia('(pointer: coarse)')`.
- **Lectores de pantalla:** los diálogos y avisos ya son HTML. Añadir `aria-live="polite"` a la caja de diálogo y a los avisos, y roles y etiquetas en los botones de las preguntas.
- **Daltonismo:** no depender solo del color en validaciones (verde y rojo acompañados de ✓ y ✗, ya pasa en las preguntas; revisarlo en cables y palancas).

---

16. Rendimiento en ordenadores de instituto

Objetivo: 30 fps estables en un portátil con gráficos integrados (Intel UHD 620) a 1080p con calidad baja.

- **Presupuesto por zona:**
    - ○ ≤ 300 *draw calls*.
    - ○ ≤ 8 luces puntuales visibles (las luces de las zonas ocultas no cuentan porque su grupo es invisible).
    - ○ Una sola luz con sombras.
- **Fusionar geometría estática:** suelos y muros de cada sala en una malla por material (`BufferGeometryUtils.mergeGeometries` de three/addons) tras colocarlos. En el Piso I, 54 baldosas y 60 muros pasarían de unas 120 llamadas a menos de 10.

- **`InstancedMesh`** para elementos repetidos (árboles del exterior, rocas, velas).
- **Calidad baja:**
  - Sin `UnrealBloomPass`: los emisivos pierden el halo, pero se mantienen.
  - Sin sombras.
  - `pixelRatio` 1.
  - Niebla más densa, para acortar `camera.far`.
- **Carga diferida y liberación de pisos** (sección 8.2).
- **Medir:** `renderer.info.render.calls` y `renderer.info.memory` en un panel de depuración (`?depurar=1`).
- Los GLB de personajes pesan 3,5 MB cada uno por sus 76 animaciones. Si la descarga pesa en el aula, crear versiones con solo las animaciones usadas (unas 12) con un script de Node que use `@gltf-transform/core` (MIT; pedir aprobación): reduciría cada una a unos 0,8 MB.

17. Pruebas

1. **Validación de contenido** (`herramientas/validar_contenido.mjs`, con `node`):
   - Cada pregunta tiene `criterio` válido.
   - `correcta` está dentro de rango.
   - Los ids son únicos.
   - Cada lección referenciada existe.
   - Todos los saberes del apéndice A están cubiertos por al menos un sello o pregunta.
   - Ejecutarla antes de cada commit.
2. **Módulos puros con `node --test`:** grafo de redes, conversiones numéricas, parser de búsqueda, validaciones de IP y subred, filtro del chat, contraste WCAG, extractor de rasgos del oráculo.
3. **Recorridos automáticos por piso:** `__torre.recorrer('piso2')`, en un `js/depuracion.js` que solo se carga con `?depurar=1`, completa un piso con las respuestas correctas y comprueba que la puerta se abre. Es la versión ordenada de los guiones usados hasta ahora.
4. **Multijugador:** 3 pestañas (sección 6).
   - Completar el piso con acciones repartidas.
   - Forzar dos acciones iguales a la vez.
   - Cerrar un invitado y luego el anfitrión.
5. **Visual:** una captura por sello (`__torre.capture`) guardada en `capturas/` y revisada a ojo. Especialmente tras oscurecer paletas nuevas.
6. **Rendimiento:** anotar `renderer.info.render.calls` por piso en el commit.

18. Publicación (GitHub Pages) y caché

- **GitHub Pages** sobre la rama `main` (carpeta raíz). Es gratuito y da HTTPS, necesario para WebXR y para WebCrypto fuera de `localhost`. URL prevista: `https://hamzaleghouiti12-sketch.github.io/DarkFantasyProyect/`. Activarlo requiere que el usuario lo apruebe; se configura en Settings → Pages.
- **Rutas:** todas las rutas del juego son relativas (`assets/…`, `js/…`); funcionará en una subcarpeta. Comprobar que ningún `fetch` usa rutas absolutas (`/js/...` solo aparece en los guiones de prueba).
- **Caché:** GitHub Pages cachea unos 10 minutos. Añadir una constante `VERSION` y usarla en:
  - `<script type="module" src="js/main.js?v=VERSION">`;
  - las importaciones dinámicas de pisos (`import('./piso2.js?v=' + VERSION)`).

Los `import` estáticos entre módulos heredan la URL sin versión: o se aceptan los 10 minutos, o se añade

un `importmap` con `scopes`. Recomendación: aceptar los 10 minutos.

- **Tamaño del repositorio:** ~22 MB hoy; con los pisos nuevos puede llegar a 40-50 MB, dentro de los límites de GitHub (1 GB por repositorio, 100 MB por archivo).
- **`Jugar.bat`** sigue sirviendo para jugar sin internet… salvo por los CDN. Opcional: copiar Three.js y PeerJS a `vendor/` para un modo sin conexión (PeerJS no funciona sin internet de todos modos).

---

19. Hoja de ruta por fases

Una "sesión" es un bloque de trabajo de 1-3 horas con Claude.

Fase	Contenido	Sesiones	Criterio de hecho
0	Motor de pisos, tipos de pregunta, estado por piso, informe, Portal de los pisos, validador de contenido, `depuracion.js`	4-5	Piso I migrado y jugable igual; pruebas en verde
1	Pisos II y III (`elegir`, `ordenar`, `clasificar`, `palancas`, preguntas numéricas)	4-5	Ambos pisos completables solo y en equipo de 3
2	Piso IV (`conectar`, `formulario`, `terminal`, islas)	3-4	Ídem, más pruebas del grafo
3	Pisos V y VII (`editorWeb`, buscador, licencias, pregones)	5-6	Ídem, más pruebas de validaciones HTML y CSS
4	Menú de opciones, accesibilidad, táctil y calidad baja	2-3	Jugable solo con teclado y en tableta
5	Pisos VIII, IX y X (`enemigos`, `casos`, `cofreCripto`)	6-7	Ídem, más revisión docente del Piso X
6	Pisos XI y XII (`bloques`, `codigo`, `placa`)	5-6	Ídem
7	Pisos VI y XIII (`modelado`, `sql`, `oraculo`)	5-6	Ídem
8	Cima, jefe final, créditos e informe final	3-4	Partida completa de principio a fin
9	Optimización, GitHub Pages y pruebas en aula	2-3	30 fps en portátil modesto; multijugador probado en 2 ordenadores reales
Total		**39-49**	

El orden de las fases 1-7 se puede cambiar según las prioridades del profesorado (p. ej., si la extraordinaria se centra en el bloque III, adelantar la fase 5).

---

20. Riesgos y decisiones abiertas

Riesgos:

1. **Público:** el Piso I es de nivel ESO; el currículo nuevo es de 2.º de Bachillerato. *Decisión del usuario:* ¿solo Bachillerato, o dos niveles de dificultad (bandera `nivel` en el contenido)?
2. **Temas sensibles (Piso X):** grooming, sextorsión y violencia digital con menores. *Mitigación:* revisión docente obligatoria, tono INCIBE, sin contenido explícito y teléfono 017 visible.
3. **Redes del instituto sin TURN:** el modo equipo puede no conectar. *Mitigación:* el juego completo funciona solo; documentarlo para el profesorado. Un TURN gratuito fiable no existe; no prometerlo.
4. **Rendimiento:** 13 pisos de KayKit en portátiles modestos. *Mitigación:* sección 16.
5. **Dependencias de CDN:** si jsDelivr cae, el juego no carga. *Mitigación:* copia local en `vendor/` (opcional).
6. **Complejidad de `main.js`:** si se añaden pisos sin la Fase 0, el proyecto se vuelve inmantenible. **No saltarse la Fase 0.**
7. **Exactitud del contenido:** normativa (RGPD, LOPDGDD, Reglamento de IA) y datos técnicos. *Mitigación:* revisar las lecciones contra el PDF del currículo y fuentes oficiales (AEPD, INCIBE, BOC); marcar en el contenido la fuente de cada dato normativo.

Decisiones pendientes del usuario:

1. Nivel (ESO/Bachillerato/ambos).
2. Aprobación de KayKit Skeletons, js-interpreter, sql.js y three-bvh-csg con three-mesh-bvh.
3. Activar GitHub Pages.
4. Informe descargable para el profesor: sí o no.
5. Si el Portal de los pisos debe estar abierto para todos o solo en modo docente.

Apéndice A · Matriz saber → piso → mecánica → criterio

Saber (currículo)	Conceptos clave	Piso	Mecánica / sello	Criterio
I.1 Almacenamiento	memoria principal/secundaria, volátil; HDD, óptico, SSD SATA/NVMe, flash, SD	II	`elegir` encargos (sello 1)	1.1
I.1	interno/externo, NAS, nube (ventajas e inconvenientes)	II	`elegir` encargos	1.1
I.1	criterios de elección (capacidad, velocidad, durabilidad, €/GB…)	II	explicaciones por opción + varita	1.1
I.1	unidades bit→PB, 1000 frente a 1024	II	`ordenar` + `numero` (sello 2)	1.1
I.1	sistemas de archivos, particiones, formatear	II	`elegir` opcional	1.1
I.1	copias completa, incremental, diferencial, 3-2-1, cifrado	II	`clasificar` 3-2-1 (sello 3) + lección de cifrado	1.1
I.2 Codificación	binario, decimal, hexadecimal, octal y conversiones	III	`palancas` (sello 1)	1.1
I.2	complemento a 2, coma flotante, booleanos	III	puerta roja + lección	1.1
I.2	ASCII, Unicode, UTF-8	III	inscripción `texto` (sello 2)	1.1
I.2	imagen 2D (píxel, resolución, profundidad, RGB; mapa de bits/vectorial; formatos; peso)	III	`numero` + `clasificar` (sello 3)	1.1
I.2	imagen 3D (mallas, texturas, STL/OBJ/FBX/glTF)	III (+VI)	"ver malla" del jugador	1.1
I.2	audio (muestreo, bits, canales, formatos, peso)	III	`numero` + `clasificar`	1.1
I.2	vídeo (fps, resolución, bitrate, códec frente a contenedor)	III	`clasificar` + varita	1.1
I.2	compresión con y sin pérdida; entornos virtuales	III	`clasificar` + lección	1.1
I.3 Redes	qué es una red; PAN, LAN, MAN, WAN	IV	lección + varita	1.2
I.3	medios (UTP/RJ45, fibra, coaxial; Wi-Fi, Bluetooth, 4G/5G, satélite)	IV	tipos de cable en `conectar`	1.2
I.3	topologías; cliente-servidor frente a P2P	IV	`conectar` estrella/malla (sello 1) + guiño al modo equipo	1.2
I.3	dispositivos (NIC, router, switch, hub, AP, repetidor, módem/ONT, PLC)	IV	colocar dispositivos en `conectar`	1.2
I.3	TCP/IP, OSI; HTTP(S), DNS, DHCP, FTP, SMTP, POP3, IMAP	IV	lección + varita + DNS en `terminal`	1.2
I.3	IP v4/v6, pública/privada, estática/dinámica, máscara, puerta de enlace, DNS, MAC	IV	`formulario` (sello 2)	1.2
I.3	configurar router y Wi-Fi (SSID, WPA2/3); ipconfig, ping, tracert; diagnóstico	IV	`terminal` (sello 3) + faro Wi-Fi	1.2

I.3	Canarias: cables submarinos, teletrabajo, brecha digital	IV	enlace de fibra entre islas + lección	1.2
I.3	RAEE, obsolescencia, ahorro energético	IV	chatarrería `clasificar` (opcional) + varita	1.2
II base	PLE, herramientas colaborativas, versiones, lenguaje inclusivo	V	lección inicial + varita	2.2, 2.3
II.1.1 Web	página/sitio, estática/dinámica; HTML; CSS; CMS; publicar; diseño y responsive	V	`editorWeb` (sellos 1 y 2) + `ordenar` publicación	2.2
II.1.2 3D	ejes, vértices, aristas, caras; primitivas; transformaciones; extrusión; booleanas	VI	`modelado` (sello 1)	2.1
II.1.2	materiales, render; STL/OBJ, laminado; programas y elección razonada	VI	sellos 2 y 3	2.1
II.1.3 RV/RA en aprendizaje	RV, RA y RM; usos educativos	VI (+Cima)	visor de realidades	2.1, 4.5
II.2.1 Licencias	derechos morales y patrimoniales, copyright, plagio, citar; copyleft, dominio público; CC; software libre, abierto, propietario, freeware, shareware	VII	`clasificar` licencias (sello 1)	2.3
II.2.2 Bulos	desinformación/información errónea, bulos, clickbait, deepfakes; verificar; burbuja de filtros; qué hacer	VII	tablón de pregones (sello 2) + burbuja	2.3, 3.3
II.2.3 Curación	fases; criterios de calidad; búsqueda avanzada; organizar (marcadores, RSS)	VII	buscador (sello 3) + `ordenar` fases	2.3
II.2.4 Accesibilidad	WCAG; lector de pantalla, subtítulos, audiodescripción, dictado, lupa, contraste; alt, contraste, tamaño, títulos	V	auditoría (sello 3) + opciones reales del juego	2.2
III.1.1 Seguridad	CIA; activa/pasiva; física/lógica; antivirus, cortafuegos, parches, contraseñas y gestor, copias, cifrado; Wi-Fi WPA2/3, Wi-Fi pública, VPN, HTTPS; permisos, cuentas sin privilegios, bloqueo	I, VIII	altar de contraseñas (I); cortafuegos y tríada (VIII)	3.1
III.1.2 Malware	virus, gusano, troyano; ransomware, spyware, adware, keylogger, rootkit, backdoor, botnet, secuestrador, criptominado; elementos atacados; vías y síntomas	VIII	`enemigos` (sello 1) + varita	3.1
III.2 Identificación	identificación/autenticación; factores; 2FA/MFA; certificado FNMT, DNIe, Cl@ve, firma; simétrico/ asimétrico; sede electrónica	I, IX	cartel 2FA (I); `cofreCripto` (IX)	3.3
III.3 Protección de datos	RGPD, LOPDGDD, AEPD; dato personal, especialmente protegidos, consentimiento a los 14, responsable; derechos; derechos digitales; huella activa/pasiva, cookies; gestión de la huella	IX	juicios (sello 1) + espejo de la huella (sello 3)	3.2
III.4 Amenazas	exposición de datos, suplantación; ciberacoso, sexting, sextorsión, grooming, doxing; phishing, smishing, vishing, ingeniería social; fraudes; prevención e INCIBE 017	I, X	pergaminos (I); chats simulados y testigo (X)	3.3
III.5 Privacidad y bienestar	privacidad en redes; identidad positiva y netiqueta; bienestar, adicción, nomofobia, FOMO; violencias sexistas digitales	X	reloj de arena (sello 3)	3.3
IV.1 Pensamiento computacional	fases; algoritmo; pseudocódigo; diagramas de flujo	XI	losas del diagrama (sello 1) + pseudocódigo junto a los bloques	4.1, 4.2

IV.2 Elementos	variables, constantes, tipos, operadores, E/S, estructuras de control, funciones, comentarios, errores y depuración	XI	`bloques` (sello 2) + depuración (sello 3)	4.1, 4.2
IV.3 Aplicaciones	lenguajes por bloques/textuales, alto/bajo nivel, compilados/interpretados; IDE; POO; apps para móvil, web y placas; ficheros; trabajo en equipo y versiones	XII	POO (sello 1), placa (sello 2), registro (sello 3)	4.3, 4.4
V.1 IA, big data, BD	IA débil/general; ML supervisado, no supervisado, refuerzo; redes neuronales, IA generativa; aplicaciones; sesgos, alucinaciones, ética, Reglamento de IA; big data (5 V, fuentes, visualización); BD relacional; SGBD y SQL; NoSQL; servidor de BD	XIII	`oraculo` (sello 1), `sql` (sello 2), 5 V y alucinaciones (sello 3)	4.3
V.2 RV/RA	RV/RA/RM/XR; hardware; elementos de una app; herramientas (CoSpaces, Unity, A-Frame); aplicaciones	Cima	`realidad` (sello de la cima)	4.5

Apéndice B · Esquemas de datos

B.1 Pregunta

```
{
  id: 'p4-07',               // único en todo el juego: p<piso>-<nn>
  concepto: 'direccionamiento',  // id de lección que la desbloquea
  criterio: '1.2',           // o ['1.2', '3.1']
  tipo: 'opcion' | 'vf' | 'numero' | 'texto' | 'orden' | 'multiple',
  texto: '¿Cuál de estas IP es privada?',
  opciones: ['8.8.8.8', '192.168.1.20', '203.0.113.5', '1.1.1.1'],  // opcion/multiple/orden
  correcta: 1,               // opcion: índice; multiple: [índices]; vf: bool; orden: [secuencia]
  valor: 931.3, tolerancia: 0.5, unidad: 'GiB',  // numero
  aceptadas: ['exfat'],      // texto (normalizado)
  explicacion: 'Las IP 192.168.x.x están reservadas para redes privadas.',
  nivel: 'bach',             // opcional si se hacen dos niveles
}
```

B.2 Lección

```
{ titulo: 'Direcciones IP', resumen: '…(para el grimorio)…', paginas: ['…', '…'], despues: ['…'] }
```

B.3 Definición de mecánica `elegir`

```
{
  mec: 'soportes', sello: 'soportes', leccion: 'almacenamiento',
  opciones: [{ id: 'hdd', texto: 'Disco duro (HDD)', modelo: 'trunk_large_A', porque: '…' }, …],
  encargos: [{ texto: 'Una fotógrafa que viaja…', correctas: ['ssd_nvme', 'sd'] }, …],
  aciertosNecesarios: 3,
}
```

B.4 Estado de la acción `mec`

```
{ piso: 'piso4', mec: 'red', paso: 'cable', de: 'isla2', a: 'switch', medio: 'utp', unica: false }
```

B.5 Progreso guardado (localStorage, versión 2)

```
{ version: 2, pisos: { piso1: { completado: true, mejorPrecision: 90, segundos: 610 } },
  criterios: { '3.1': { a: 12, f: 3 } }, opciones: { texto: 1.25, contraste: false, … } }
```

Apéndice C · Catálogo de piezas KayKit disponibles (ya descargadas, en `assets/_descargas/`)

- **Dungeon Remastered (203):**
  - Muros y variantes: normal, agrietado, arqueado, con ventana, con reja, con andamio, esquinas, medio muro, en pendiente, con estanterías y con puerta.
  - Suelos: baldosa grande y pequeña, con rocas, con hierba, rota, con rejilla, con pinchos, de madera, de tierra, cimientos.
  - Escaleras: normal, estrecha, ancha, con muros, de madera.
  - Estructura: columna, pilar, pilar decorado, barreras.
  - Mobiliario: mesas (larga, mediana, pequeña, con mantel, rotas, decoradas), sillas, taburetes, estanterías, camas.
  - Contenedores: cofres (normal y dorado), baúles (grande, mediano, pequeño), barriles, barriletes, cajas.
  - Iluminación: velas, antorchas (en mano y de pared).
  - Objetos: botellas, platos, monedas, llaves y llavero, espada y escudo.
  - Decoración: estandartes de 6 colores y 5 motivos, escombros.
- **Halloween Bits:** árboles muertos y pinos, tumbas y lápidas, cripta, ataúdes, vallas y verjas, farolas y faroles, calabazas, santuario, calaveras y huesos, caminos, bancos, arco con verja.
- **Medieval Hexagon:**
  - Edificios en 4 colores y neutros: casas, torres, castillo, iglesia, taberna, herrería, mercado, molinos, pozo, mina, cuarteles, aserradero, puente.
  - Naturaleza: árboles, colinas, montañas, rocas, plantas de agua.
  - Baldosas hexagonales: hierba, costa, ríos, caminos.
  - Objetos: carretillas, cajas, sacos, dianas, tiendas, banderas, murallas.
- **Adventurers:** Knight, Barbarian, Mage, Rogue, Rogue_Hooded (76 animaciones cada uno) y armas y objetos (libros de hechizos, bastón, varita, escudos, jarras, bombas de humo).

Para usar una pieza nueva: copiarla a la carpeta del proyecto correspondiente, añadir su nombre a la lista de `cargarPiezas` de su zona y aplicarle la paleta oscura de su pack.

Apéndice D · Mensajes de red (actuales y nuevos)

`t`	Estado	Frecuencia	Notas
`hola`, `bienvenida`, `rechazo`, `sala`, `perfil`, `empezar`	existentes	eventos	sala
`pos`	existente	12 Hz	añadir `lleva` (objeto transportado)
`hechizo`	existente	evento	generalizar el objetivo (`diana` por id)
`acc` / `ev`	existentes	evento	nuevo tipo genérico `mec`
`chat`, `latido`	existentes	evento / 1 Hz	
`ene`	**nuevo**	8 Hz	lista de enemigos (solo el anfitrión)
`snap`	**nuevo**	al entrar en un piso	instantánea del estado del piso
`pieza`	**nuevo** (opcional)	10 Hz	forja cooperativa

Apéndice E · Lista de comprobación por piso (definición de hecho)

1. [ ] Contenido: lecciones, preguntas (≥ 12, con `criterio`), pistas para cada sello, `memoria` narrativa y `sellosRotos`.
2. [ ] `node herramientas/validar_contenido.mjs` en verde.
3. [ ] Pruebas de los módulos puros nuevos (`node --test`) en verde.
4. [ ] Sala construida con `crearSalaDeTorre`; ≤ 300 *draw calls*; ≤ 8 luces.
5. [ ] Los 3 sellos se pueden completar **solo**, con `__torre.recorrer`.
6. [ ] En equipo de 3: sellos repartidos, una acción duplicada a la vez y la desconexión de un invitado.
7. [ ] Capturas de cada sello revisadas, y la paleta oscura revisada si hay piezas nuevas.
8. [ ] Música del piso y efectos nuevos; tecla M operativa.
9. [ ] Opciones de accesibilidad respetadas (reducir movimiento, tamaño de texto).
10. [ ] Informe por criterios con datos del piso.
11. [ ] Commit en español con `Co-Authored-By`; subir **solo si el usuario lo pide**; excluir los archivos del usuario.

Apéndice F · Comandos útiles

```
# servidor local (o preview_start 'juego-torre')
python -m http.server 8750 --bind 127.0.0.1

# comprobar sintaxis de todos los módulos
for f in js/*.js; do node --check "$f" || echo "FALLO $f"; done

# regenerar paletas oscuras
python herramientas/oscurecer_paleta.py

# medir el proyecto
wc -l js/*.js | sort -n
du -sh assets/modelos/*

# bajar un pack de KayKit (solo tras la aprobación del usuario)
curl -sL -o pack.zip https://github.com/KayKit-Game-Assets/<REPO>/archive/refs/heads/main.zip
```

Guiones de consola en el navegador (con el juego abierto):

```
// invalidar la caché de los módulos antes de recargar
for (const f of ['main','ui','red']) await fetch(`/js/${f}.js`, {cache:'reload'}); location.href='/?

// saltar la portada y el prólogo
document.getElementById('btn-solo').click();
// … luego pulsar el botón de .story-actions varias veces

// vaciar diálogos
const T = __torre; const key = (c,k) => dispatchEvent(new KeyboardEvent('keydown',{code:c,key:k}));
while (T.ui.open.dialogue) { key('KeyE','e'); await new Promise(r => setTimeout(r, 200)); }

// ir a una zona y avanzar el juego
T.irA('piso1'); T.step(120);
```

Fin del documento.

Preguntas vivas (capa común a las ~220 preguntas)

- ⚡ **Rayo:** anillo de 12 s (22 s en las escritas). Si se acierta antes de que se vacíe: +5 de saber. Ir lento no tiene castigo.
- 🔥 **Racha:** contador en el HUD. Desde 3 aciertos seguidos suma +2, +4… (hasta +10). A los 5 y a los 10 sale un aviso, y cada 3 seguidos el jugador y Aldric celebran. Un fallo la pone a 0.

- **Reacciones:** frases variadas en lugar de «¡Correcto!» / «No exactamente.» (`VIVAS` y `ANIMOS` en `ui.js`).
- **Efectos:**
  - la carta entra con un salto y las opciones aparecen escalonadas;
  - al acertar: brillo y confeti, y chispas 3D alrededor del jugador;
  - al fallar: la carta tiembla y la cámara da una sacudida leve.
  - La opción «Reducir el movimiento» y `prefers-reduced-motion` desactivan el confeti, la sacudida y los saltos.
- **Código:**
  - `ui.prepararCarta` y `ui.reaccionar`;
  - en `main.js`, `ui.onRespuesta(ok, {ms, limite})` devuelve `{racha, rapido, bonusRacha}`.
- **El mundo responde:** `zona.alResponder?.(ok, racha)`.
  - Las salas de la torre (pisos II–XIII) lo heredan de `sala-torre.js`: las antorchas se avivan doradas y echan chispas al acertar (más fuerte con más racha) y bajan a un azul frío al fallar.
  - En la Cima, durante el duelo, Morvath acusa cada acierto (golpe y chispas doradas) y se crece con cada fallo (humo violeta).
  - Un piso puede sustituir `alResponder` por su propia reacción.
- **Series como asaltos:** `serieDePreguntas` pone `ui.barraSerie`.
  - Encima de la pregunta aparece una barra de sello con un segmento por prueba: los aciertos la van rompiendo y el segmento actual late.
  - Al fallar, el segmento aguanta (tiembla) y Aldric despide con una frase variada.
  - Se quitó el doble sonido de acierto: ya lo pone `onRespuesta`.
- **Varita cargada por la racha:** con 3 aciertos seguidos el hechizo sale *potenciado* (×1.6) y con 5, *desatado* (×2.2).
  - Es más grande, con más estela, una explosión con destello blanco y una sacudida leve.
  - `spells.cast(..., potencia)`; la potencia viaja en el mensaje `hechizo` para que los compañeros lo vean igual.
- **Piso I · corro de pergaminos vivo:** cada falso que arde acelera el corro (`world.ritmo` +0.45) y lo hace bailar en ola. Un disparo a un legítimo lo dispersa del susto (`world.dispersar`). Todos se mueven igual, para no delatar cuál es falso, y avisos dicen cuántos quedan.
- **Guardianes con personalidad (pisos II–VII):**
  - Al acercarse el jugador a menos de 4,5 m, el guardián se para, se gira hacia él, hace un amago de ataque y suelta una pulla en un bocadillo. Las pullas se ríen de los errores que enseña la torre: «¿Vuestra contraseña es 1234?».
  - Vuelven a hacerlo cada 12–18 s como mucho.
  - Al ser desterrados, se despiden con humor («Error 404: guardián no encontrado…»).
  - Todo esto es local a cada jugador, como la patrulla.
- **Corrección:** el susto de los pergaminos ahora abre el corro +3,6 m y dura unos 2 s.
- **Balance del piso con estrellas:** al subir de piso aparece una carta durante 6 s, sin bloquear el juego.
  - Muestra de ★ a ★★★, el % de aciertos, la mejor racha, las respuestas ⚡ rápidas y los minutos.
  - Hacen falta ★★★ con un 85 % o más y al menos 3 aciertos; ★★ con un 60 % o más.
  - Datos en `state.balance` (los alimenta `onRespuesta`); se pinta con `ui.balancePiso`.
  - Invita a rejugar un piso para sacar las tres.
- **Aprender se celebra:** al aprender un concepto, una página entra girando a la derecha («Nuevo concepto en el grimorio», con el título) y salen chispas azules alrededor del jugador. Sustituye al

aviso plano, también en los pisos II, IV, VII, VIII y X.

- **Aldric gesticula al hablar:** con `ui.onPagina`, en cada página hace un gesto y, si exclama, a veces lo celebra.
- **Piso I · reloj de fuerza bruta en el altar:** cada cristal ofrecido dice cuánto tardaría un ordenador en adivinar esa contraseña.
  - Ejemplos: «123456789», al instante; «maria2009», unos 2 segundos; «Gato-Luna-Tostada-47!», más que la edad del universo.
  - Lo calcula `js/mecanicas/fuerza.js`, con 10^{10} intentos/s y el atacante probando primero lo fácil: las más usadas, el diccionario y palabra+número; después, la fuerza bruta.
  - Pruebas en `fuerza.test.mjs`; en total pasan 68.
- **Rachas en equipo:** al llegar a 3, 5, 10… seguidas se envía un `hito` (`red.enviarHito`).
  - El anfitrión le pone el nombre, como en el chat, para que nadie se haga pasar por otro, y lo reenvía.
  - El resto ve un aviso («🔥 Ana lleva 5 aciertos seguidos. ¡Imparable!») y un bocadillo sobre ese compañero.
- **Las criaturas reaccionan a las respuestas:**
  - Los guardianes a menos de 14 m se burlan (animación *Taunt*, «¡Ja! ¡Fallo!») cuando el jugador falla. Cuando acierta, lo acusan (*Block_Hit*, «¡Ay!», un instante aturdidos).
  - Al ver al jugador, los guardianes hacen *Taunt* en vez del amago de ataque.
  - En el Piso XII, las criaturas forjadas vitorean (*Cheer*, escalonado) cada acierto y se cubren con cada fallo.
  - Se encadena con la reacción de las antorchas de la sala.
- 🪄 **Comodín de Aldric:** en las preguntas de 3 o más opciones aparece un botón (o la tecla **C**) que tacha todas las falsas menos una.
  - Se tiene 1 al empezar y cada piso nuevo garantiza al menos 1.
  - Cada racha múltiplo de 5 regala otro, hasta un máximo de 3.
  - Las opciones tachadas no se pueden elegir. Datos: `ui.comodines` y `ui.onComodin`.
- **Corrección:** el altar de cristales y los pergaminos del Piso I ya cuentan en el balance con estrellas. Antes solo contaba el cartel y el piso no podía dar ★★★.
- **Regresión del Piso I (con preguntas simuladas):** lección, cartel, altar (un fallo y un acierto), pergaminos, puerta y subida con balance (5/6, ★★) y llegada al Piso II. Todo bien.
- ✧ **La chispa de Morvath** (`js/chispa.js`): en los pisos I–XIII, cada 2,5–4 minutos de juego (sin contar diálogos ni ventanas) aparece un fuego fatuo a 5 m del jugador.
  - Revolotea en lazos y huye un poco del jugador, dentro de los límites de la sala.
  - Al alcanzarlo sale una pregunta relámpago de repaso. Si se acierta da +15 de saber y, la mitad de las veces, un 🪄 comodín.
  - Si nadie lo caza, parpadea y se apaga a los 25 s.
  - No aparece en la Cima ni fuera de la torre, ni antes de haber aprendido algo. Es local a cada jugador y no usa luces nuevas.
- **Trofeos en la pantalla final:** 🔥 mejor racha de la partida, ★ estrellas sumadas de todos los pisos superados (incluido el último, «x / máximo») y ✧ chispas cazadas. Datos en `state.trofeos`; el cálculo de estrellas está en `estrellasDe()` de `ui.js`, que lo comparten el balance y el final.
- **La racha suena:** `sonido.acierto(racha, rapido)` sube un peldaño de escala pentatónica con cada acierto seguido (tope: una décima por encima). Con ⚡ añade un destello agudo y a las 5 y 10 seguidas suena un acorde. Las demás llamadas a `acierto()` no cambian.
- **Corrección de diseño (móvil):**
  - La carta de balance ya no usa `translateX(-50%)`, porque la animación de entrada se lo quitaba y la carta daba un salto lateral. Ahora va centrada con márgenes automáticos y cabe en 375 px.

    - La página de concepto nuevo mide como mucho el ancho de la pantalla menos 32 px y en móvil sale arriba.

- **Piso X · las sombras se alimentan de los errores:** cada acierto las encoge un 10 % (mínimo 55 %), con un golpe y chispas claras, y cada fallo las hace crecer un 8 % (máximo 130 %), con transición suave. Refuerza el mensaje del piso: plantar cara (saber qué hacer) debilita el acoso.
- **Piso XIII · el cielo responde:** cada acierto lanza estrellas fugaces por la cúpula (1 a 3 según la racha) y hace brillar las estrellas. Cada fallo hace parpadear el orbe del oráculo en rojo un instante.
- **Piso IX · la balanza pesa vuestras respuestas:** se inclina hacia un lado con cada acierto (chispas doradas) y hacia el otro con cada fallo, y vuelve poco a poco al centro.
- **Piso XI · el autómata sigue la partida:** con cada acierto da un saltito con la cabeza y gira la llave muy deprisa; con cada fallo niega con la cabeza. El ojo ahora es hijo de la cabeza.
- **Piso I también responde:** sus antorchas (de `world.js`, que no usa la sala común) se avivan doradas al acertar y bajan a azul al fallar, igual que en las salas de los demás pisos (`world.reaccionar`). Con esto, los 13 pisos y la Cima reaccionan a cada respuesta.
- **Prueba en equipo real (2 pestañas, PeerJS):** los `hito` llegan en las dos direcciones, del invitado al anfitrión («🔥 Beto lleva 3 aciertos seguidos») y del anfitrión al invitado (3 y 5, «¡Imparable!»), con el nombre puesto por el anfitrión.
- 🪄 **Comodín también en las preguntas escritas:**
    - Números: un rango redondo que contiene la respuesta, sin bajar de 0.
    - Palabras: la primera letra y la longitud.
    - Ordenar: Aldric coloca la primera pieza (también con **C**).
    - En las de escribir, la tecla C no activa el comodín para no interferir al teclear.
- **Informe para el profesor:** el nuevo bloque «Esta sesión» muestra la mejor racha, las estrellas, las preguntas relámpago acertadas y los **comodines usados**, con una nota sobre cómo afectan los comodines a los porcentajes.
    - Se corrige la nota antigua que decía que faltaban pisos: ya están todos los criterios, del 1.1 al 4.5.
    - Nueva prueba en `informe.test.mjs`; en total pasan 69.
- ⚠ **Error grave corregido (ya estaba subido en 9ebaf31):** el Piso VIII no cargaba. Tenía `const destino` (vector de las rondas) y `function destino()` (la guía de misión) en el mismo ámbito, y el navegador lo rechaza como SyntaxError.
    - El vector se renombra a `rumbo`.
    - `node --check` no lo detectaba porque, sin package.json, trata los .js como scripts. Nueva prueba `herramientas/pruebas/sintaxis.test.mjs`: comprueba todos los módulos de js/ como módulos ES. En total pasan 70.
- **Barrido final:** los pisos I–XIII y la Cima cargan y reaccionan a aciertos y fallos sin errores.

Personaje nuevo: el Monstruo (2026-10-19)

Personaje jugable generado a partir de una imagen. Todo está en `herramientas/monstruo/`, y el proceso explicado en `Como_se_hizo_el_monstruo.pdf` (3 páginas).

1. `generar_3d.py`: TRELLIS en el Space gratuito de Hugging Face (trellis-community/TRELLIS). Usa el protocolo de cola de Gradio 5 (`queue/join` + SSE) con `urllib`: abre sesión (fn 4), quita el fondo (fn 8) y genera el GLB (fn 11). Sin instalar nada y con pocos MB de RAM en local.
2. `glb_a_obj.py`: convierte el GLB a OBJ + MTL + PNG (solo numpy). Resultado: `monstruo.obj` con 9.993 vértices y 15.754 caras.
3. `analizar.py`: corta la malla en rodajas para colocar las articulaciones.
4. `construir_rig.py`:
    - 13 huesos, más `handslotr`/`handslotl` para la varita;
    - pesos de piel por regiones con fundido suave en las articulaciones;

- 16 animaciones procedurales con nombres de KayKit (Idle, Walking_A, Running_A, Jump_Idle, Cheer, Hit_A/B, Interact, Spellcast_Shoot, Spellcasting, Unarmed_Melee_Attack_Punch_A, Death_A, Dodge_Backward, Sit_Floor_Idle, Sit_Floor_StandUp, Taunt).
- Salida: `assets/modelos/monstruo.glb` (0,95 MB).

5. Juego:
   - `personajes.js`: entrada `monstruo`, con `paleta: null` porque trae su propia textura.
   - `armas.js`: `natural: true` y quinto tipo de ataque, `cabezazo` (2,1 m, 90°, 0,65 s); efecto en `ataques.js` y sonido en `audio.js`.
   - `visor.html`: visor de poses.
6. Ojo al capturar poses: pintar varias poses en el mismo fotograma deja triángulos estirados, porque la GPU reutiliza los huesos de la pose anterior. Hay que capturar una pose por fotograma.